Kingdom of Saudi Arabia
Ministry of Education
University of Business & Technology
Bachelor of Science in Software Engineering

NeuroPix: AI-Enhanced Image Processing and Management Web Application

Abdulrahman Alguydi (YB0062)
Mohammed Dafterdar (VA0056)
Saud Almansour (VA0040)

Supervisor
Dr. Faisal Arafsha

University of Business & Technology

2026

A SENIOR PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF SOFTWARE ENGINEERING, COLLEGE OF ENGINEERING, UNIVERSITY OF BUSINESS & TECHNOLOGY IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

Acknowledgments

We thank our supervisor, Dr. Faisal Arafsha, for his guidance and feedback throughout the development of NeuroPix. We also appreciate the support provided by the Department of Software Engineering at the University of Business and Technology. Finally, we thank our families and colleagues for their patience and encouragement during the project.

Table of Contents

Acknowledgments ..... **i**
Table of Contents ..... **ii**
List of Tables ..... **iv**
List of Figures ..... **v**
Abstract ..... **vi**
List of Abbreviations ..... **vii**
Chapter 1 Introduction **1**
1.1 Problem Statement and Significance 1
1.2 Proposed Solution 2
1.2.1 Aim and Objectives 2
1.2.2 Project Deliverables 2
1.3 Project Domain and Intended Users 3
1.4 Scope and Limitations 3
1.5 Definitions 4
Chapter 2 Background and Related Work **5**
2.1 Image-Editing Background 5
2.2 Related Systems 5
2.3 Comparative Analysis 6
2.4 NeuroPix Differentiation 6
Chapter 3 System Analysis **7**
3.1 Requirements Identification Method 7
3.2 Stakeholders and User Characteristics 8
3.3 Functional Requirements 8
3.4 Nonfunctional Requirements 9
3.5 Use-Case Model and Descriptions 9
3.6 Current System Workflow 11
3.7 Data Requirements and Business Rules 12
3.8 Software and Hardware Requirements 12
Chapter 4 System Design **13**
4.1 Design Approach 13
4.2 High-Level Architecture 13
4.3 Frontend Navigation and Access Boundaries 14
4.4 Backend and API Design 15
4.5 Standard Image-Processing Design 16
4.6 AI Prompting and Provider Routing 17
4.6.1 OpenAI pre-prompt construction 18
4.6.2 Local model request and response 20
4.7 Database Design 21
4.7.1 Full database schema 22
4.7.2 Connecting to MySQL 23
4.7.3 Ownership checks in SQL 24
4.8 Image Storage and Lifecycle 25
4.8.1 Uploading the image versions to S3 25
4.9 Security Design 26

4.9.1 Server configuration and secret keys 26
4.9.2 Environment configuration example 27
4.10 Design-Level Interaction Sequences 28
4.11 Deployment and Reverse-SSH Design 32
4.12 Development Decisions and Trade-Offs 33
Chapter 5 Implementation 34
5.1 Technology Stack and Environment 34
5.2 Frontend and Authentication Implementation 34
5.3 Upload and Standard Processing Implementation 35
5.4 AI Editing Implementation 35
5.5 Persistence, Dashboard, and Gallery Implementation 36
5.6 Production Deployment and Continuous Delivery 36
5.7 Implemented Input and Output Screens 37
Chapter 6 Testing and Evaluation 40
6.1 Testing Strategy 40
6.2 Live Test Environment and Controlled Data 40
6.3 Evidence and Completion Rules 41
6.4 UC-T01: Register a New User 41
6.5 UC-T02: Login and Access a Protected Page 44
6.6 UC-T03: Upload and Apply a Standard Edit 46
6.7 UC-T04: Edit with OpenAI 49
6.8 UC-T05: Edit with Local Stable Diffusion 52
6.9 UC-T06: Verify and Delete a Gallery Image 55
6.10 Results Summary and Evaluation 57
Chapter 7 Conclusion and Future Work 58
7.1 Achievement of the Project Aim 58
7.2 Critical Evaluation and Lessons Learned 58
7.3 Recommended Six-Month Continuation 58
7.4 Final Conclusion 58
References 59

List of Tables

Table 1.2.2: Project deliverables and their evidence. 2
Table 2.3: Comparison of NeuroPix with selected related systems. 6
Table 3.1-1: Sources used to recover and confirm the final requirements 7
Table 3.1-2: Principal system inputs, processing, and outputs. 7
Table 3.3: Grouped functional requirements. 8
Table 3.4: Nonfunctional requirements and current design response. 9
Table 3.5: Compact descriptions of the six principal use cases. 10
Table 3.7-1: Users entity data dictionary. 12
Table 3.7-2: Images entity data dictionary. 12
Table 4.4: Implemented backend route groups. 15
Table 4.9: Security controls and remaining boundaries. 26
Table 4.12: Principal development decisions and trade-offs. 33
Table 5.1: Implemented technology stack. 34
Table 6.1: Automated test result. 40
Table 6.2: Live use-case test environment. 40
Table 6.4: UC-T01 execution result. 41
Table 6.5: UC-T02 execution result. 44
Table 6.6: UC-T03 execution result. 46
Table 6.7: UC-T04 execution result. 49
Table 6.8: UC-T05 execution result. 52
Table 6.9: UC-T06 execution result. 56
Table 6.10: Consolidated verification results. 57

List of Figures

Figure 1.1: Fragmented and unified editing workflows. 1
Figure 3.5: NeuroPix use-case model. 9
Figure 3.6: Authenticated editing activity. 11
Figure 4.2: NeuroPix system architecture. 13
Figure 4.3: Frontend navigation and access. 14
Figure 4.5: Standard processing order. 16
Figure 4.6: AI prompt and provider routing. 17
Figure 4.7: NeuroPix data model. 21
Figure 4.8: Image storage lifecycle. 25
Figure 4.10-1: Authentication sequence. 28
Figure 4.10-2: Standard-edit sequence. 29
Figure 4.10-3: AI-provider sequence. 30
Figure 4.10-4: Gallery sequence. 31
Figure 4.11: Production deployment and reverse SSH. 32
Figure 5.7-1: Dashboard and Studio upload. 37
Figure 5.7-2: Standard and OpenAI controls. 38
Figure 5.7-3: AI comparison and gallery. 39
Figure 6.4-1: UC-T01 registration sequence. 41
Figure 6.4-2: UC-T01 registration evidence. 43
Figure 6.5-1: UC-T02 login and protected-access sequence. 44
Figure 6.5-2: UC-T02 login evidence. 45
Figure 6.6-1: UC-T03 upload and Standard-edit sequence. 46
Figure 6.6-2: UC-T03 Standard-edit evidence. 48
Figure 6.7-1: UC-T04 OpenAI-edit sequence. 49
Figure 6.7-2: UC-T04 OpenAI-edit evidence. 51
Figure 6.8-1: UC-T05 local Stable Diffusion sequence 52
Figure 6.8-2: UC-T05 local-model evidence 54
Figure 6.9-1: UC-T06 gallery verification and deletion sequence. 55
Figure 6.9-2: UC-T06 gallery evidence. 57

Abstract

Image editing and artificial intelligence (AI)-assisted changes often require users to move files between separate tools. NeuroPix is a web application that combines standard controls, guided AI editing, before-and-after comparison, and a personal gallery. Structured user choices become prompts for OpenAI or locally hosted Stable Diffusion, while Amazon Web Services stores the images and application data. The result is a unified workflow that reduces repeated uploads, exports, and management of separate image copies. The application is available at https://neuropix.me.

Keywords: image processing, AI-assisted image editing, Flask, OpenAI, Stable Diffusion, web application

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration and Continuous Delivery
EC2	Elastic Compute Cloud
GPU	Graphics Processing Unit
HLD	High-Level Design
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
RDS	Relational Database Service
S3	Simple Storage Service
SQL	Structured Query Language
SRS	Software Requirements Specification
SSH	Secure Shell
TLS	Transport Layer Security
UI	User Interface

Chapter 1 Introduction

1.1 Problem Statement and Significance

Combining conventional adjustments with AI-assisted changes can involve moving an image between tools. For example, a user may crop an image in one editor, upload it to an AI service, compare the result with the source, and organize the saved files manually. This creates extra upload, export, and file-selection steps. NeuroPix addresses this workflow by keeping the source, editing controls, comparison, and saved results together.

For users with basic editing needs, choosing tools and tracking file versions can take attention away from the image itself. An open-ended AI prompt also requires the user to describe both the desired change and what should remain unchanged. We wanted to reduce these steps through a focused browser interface and guided editing fields.

We developed NeuroPix for users with basic browser skills, without requiring professional editing experience. Users sign in to upload an image, apply Standard or AI edits, compare and download results, and manage their gallery in the browser. Figure 1.1 compares this workflow with using separate tools.

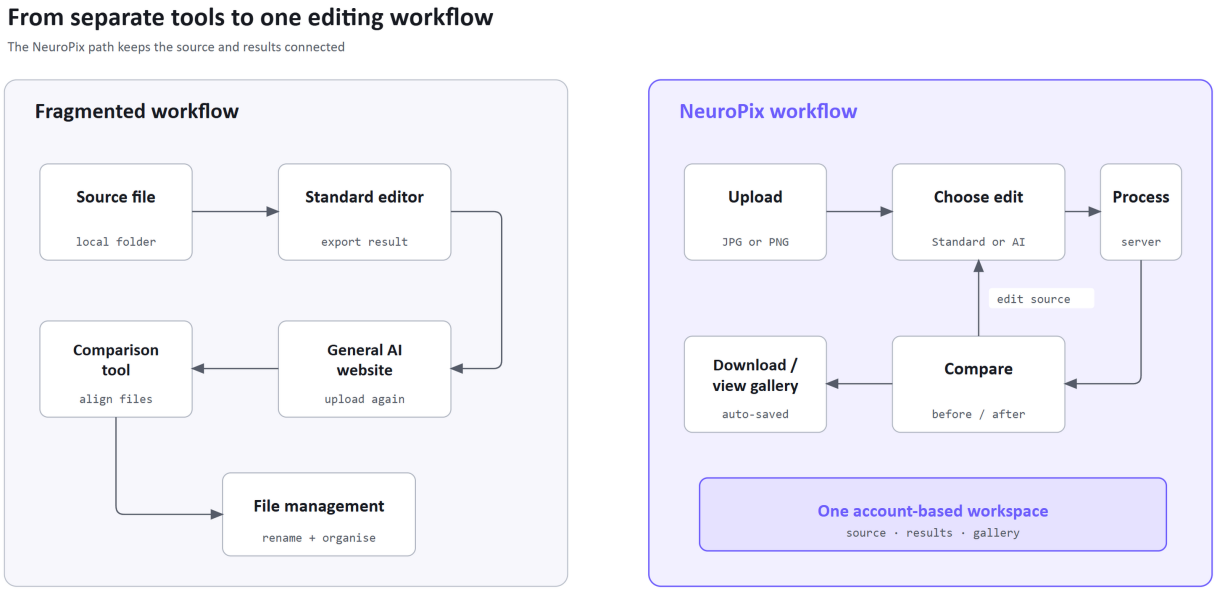


Figure 1.1: Fragmented and unified editing workflows.

The workflow requires file validation, account-specific gallery access, and coordination between image storage and database records. It must also handle AI services that may be slow or unavailable.

The project demonstrates how deterministic image operations and generative editing can share a simple account-based workflow. It also gives the team practical experience connecting web development, image processing, AI services, and cloud deployment.

1.2 Proposed Solution

NeuroPix is an authenticated web application for uploading an image, applying Standard or AI edits, comparing the output with the original, downloading it, and managing saved results in a personal gallery. Pillow handles Standard edits, while AI edits use OpenAI by default or locally hosted Stable Diffusion as the self-hosted option. For OpenAI requests, the interface separates generative modification, background manipulation, and enhancement into three fields, then NeuroPix adds field-specific context so users do not need to write one combined prompt. Amazon S3 stores the image files, and MySQL stores the related account and image metadata [1]. The SRS and HLD document the earlier requirements and design [2], [3].

1.2.1 Aim and Objectives

The main aim is to design, implement, deploy, and evaluate a web application that combines conventional image processing, AI-assisted editing, visual comparison, and user-specific image management.

The project objectives are:

- To provide an authenticated browser workflow for uploading and editing supported JPG and PNG images without requiring a desktop editor.
- To provide Standard controls and purpose-specific AI-editing fields, using OpenAI by default and self-hosted Stable Diffusion as the local alternative.
- To store original and processed image objects in Amazon S3 while storing account and image metadata in MySQL.
- To support before-and-after comparison, downloading, dashboard summaries, personal gallery management, and re-editing of original or edited images.
- To protect account and user-owned records through password hashing, parameterized queries, session checks, and ownership filtering.
- To evaluate the principal workflows through a compact automated test summary and six live use-case tests.

1.2.2 Project Deliverables

Table 1.2.2 lists the project deliverables and the evidence for each.

Table 1.2.2: Project deliverables and their evidence.

Deliverable	Evidence
Deployed web application	Authenticated dashboard, Studio, comparison, download, and gallery workflows available through the deployed site
Standard and AI editing	Pillow processing, OpenAI image editing, and the self-hosted Stable Diffusion integration
Persistence layer	Amazon S3 original/processed objects and MySQL user/image metadata
Design and deployment material	Current architecture, database, storage, sequence, and deployment diagrams based on the implementation
Verification material	Automated API/security summary plus six live use-case tests with sequence diagrams and screenshots
Final report	Project design, implementation, testing, and conclusions

1.3 Project Domain and Intended Users

NeuroPix combines conventional raster processing and generative image editing in an authenticated web application with cloud file storage. Users work in the browser, while the server handles processing and storage.

The intended users want straightforward adjustments or clear AI transformations without installing specialist software. They may be students, casual creators, or people trying AI editing. They need only common browser skills: creating an account, selecting files, adjusting controls, entering short descriptions, and choosing saved results.

Users access OpenAI and locally hosted Stable Diffusion through the same browser interface. They do not need to install the local model on their own devices, because it runs on the team's computer.

1.4 Scope and Limitations

The implemented scope includes account registration, session-based login, JPG/PNG upload validation, ten Standard editing controls, OpenAI prompt-based editing, self-hosted Stable Diffusion image-to-image processing, visual comparison, result download, dashboard summaries, and a user-filtered gallery. Gallery items can be reloaded for editing, downloaded as the original upload or saved edited image, and deleted. Original and processed image files are kept in separate S3 locations, while the database stores ownership and editing metadata [1].

The project does not attempt to reproduce a professional layer-based editor. Video, vector graphics, RAW photography formats, animated images, real-time collaboration, social sharing, public image feeds, mobile-native applications, and foundation-model training are outside the scope. It also does not claim enterprise-scale availability or performance because those properties have not been measured.

Several limitations follow from the chosen design. Generative results are probabilistic and may change details that the user did not mention. Each provider also adds operational dependencies: OpenAI processing depends on valid credentials, internet access, service availability, latency, and account limits, while the local provider depends on the RTX 3080 computer, its Docker service, and the reverse connection. Because processing is synchronous, a slow AI request keeps the user waiting until it finishes. Operations involving both S3 and MySQL also lack complete transactional rollback, so a partial external failure may require manual correction.

1.5 Definitions

Term	Meaning in NeuroPix
Standard edit	A deterministic image operation performed with Pillow, such as crop, rotation, brightness, contrast, blur, sharpness, or grayscale
AI edit	A prompt-based image transformation processed by OpenAI or the self-hosted Stable Diffusion service
Structured pre-prompting	Server-side addition of field-specific rules, preservation guidance, and source-image context around the user's OpenAI request
Image-to-image generation	Generative processing that uses an uploaded image as the visual starting point for a new result
Object storage	Storage of image files as objects in Amazon S3 rather than as binary data in MySQL
Metadata	Database information about a user-owned image, including object locations, edit type, and related attributes
Session	Signed session data identifying the logged-in user and the image staged for editing
Comparison slider	Interface control that reveals portions of the original and processed image in the same viewing area

Chapter 2 Background and Related Work

2.1 Image-Editing Background

NeuroPix combines deterministic and generative editing. Its Standard mode uses Pillow for operations such as crop, rotation, brightness, contrast, saturation, detail adjustment, and partial grayscale [1], [4]. These controls use measurable settings and follow a fixed processing order. Generative editing interprets a text instruction and may alter content beyond the requested region. Latent diffusion reduces the computational requirements of image generation by operating in a learned latent space [5].

Hosted and self-hosted AI differ in where generation runs and who maintains it. A managed service such as OpenAI provides image editing through an API [6]. A self-hosted model runs on hardware maintained by the application team, giving the team control over the service but responsibility for its resources and availability.

2.2 Related Systems

Adobe Photoshop is a professional image editor with detailed selection, layer, compositing, correction, and generative tools. Its generative features can add, remove, or modify content within a mature editing environment [7]. These tools support a wider range of professional tasks than the focused editing workflow we set out to build.

Canva provides a browser-based design environment with photo editing integrated into a broader content-creation workflow [8]. Its templates and design tools extend beyond the image-editing tasks in this project. We chose a smaller scope for NeuroPix so we could concentrate on combining Standard and AI editing with comparison and a personal gallery.

2.3 Comparative Analysis

Table 2.3: Comparison of NeuroPix with selected related systems.

Category	Adobe Photoshop	Canva	NeuroPix
Main problem and users	Detailed professional image creation and manipulation	Accessible visual content creation for a broad user base	Focused Standard and AI image editing for authenticated browser users
Access and design approach	Desktop-centered professional workspace with extensive tools	Browser-based template and design workspace	Browser-based upload, edit, compare, download, and gallery path
Editing and output	Deep raster editing, layers, selections, and generative features	Photo edits within designs and templates	Ten Standard controls, OpenAI editing, self-hosted Stable Diffusion, and direct image output
Guidance and management	Professional controls and workflow features	Guided design interface and asset organization	Purpose-specific prompt fields, automatic OpenAI context, comparison, and personal gallery
Relevant limitation	More complexity than this project's intended task flow	Broader design scope than the project's focused image workflow	Narrower toolset, synchronous processing, and environment-dependent AI services

2.4 NeuroPix Differentiation

NeuroPix focuses on a straightforward workflow. The user signs in, uploads one image, chooses an editing mode, reviews the result, and either downloads it or manages it in the gallery. The interface keeps the source and processed images together. Compared with a general-purpose AI website that relies on one prompt box, NeuroPix separates modification, background, and enhancement instructions. It automatically adds scope and preservation guidance [1], allowing users to describe the desired change without writing all of that context. This guidance simplifies prompt entry without reducing the selected model's capabilities.

Chapter 3 System Analysis

3.1 Requirements Identification Method

We checked the final implementation against our SRS and HLD. The routes, service modules, database schema, and interface showed which requirements were implemented, while the deployment files identified the operating requirements. Where the planning documents differed from the finished system, this chapter describes the implemented behavior [1]-[3].

Table 3.1-1: Sources used to recover and confirm the final requirements.

Evidence source	Purpose and use
Flask routes and service modules	Identified authentication, upload, processing, download, gallery, and provider behavior implemented by the backend
Frontend files and live interface	Confirmed page access rules, visible controls, browser validation, feedback, comparison, and navigation
Database schema and persistence code	Confirmed entities, ownership links, stored fields, queries, and deletion behavior
Deployment files and live environment	Confirmed EC2 hosting, Nginx, HTTPS, RDS, S3, automated restart, health checking, and the reverse-SSH path
SRS and HLD	Recorded project intent and earlier design. The implementation took precedence where they differed

Table 3.1-2 identifies the main inputs and expected outputs for each editing mode.

Table 3.1-2: Principal system inputs, processing, and outputs.

Mode	Inputs	Processing	Expected output
Standard editing	Supported image and numeric settings for crop, rotation, brightness, contrast, exposure, saturation, blur, sharpness, and grayscale	Validation followed by the fixed Pillow pipeline	Processed JPEG, comparison view, downloadable result, and saved metadata
OpenAI editing	Supported image plus any combination of generative, background, and enhancement instructions	Structured prompt construction, output-size selection, hosted image request, and decoded result	AI-edited PNG, comparison view, download, and gallery record
Local AI editing	Supported image, provider selection, and nonempty user text	Combined prompt sent to the self-hosted Stable Diffusion image-to-image service	Locally generated image, comparison view, download, and gallery record when the service and reverse connection are available

3.2 Stakeholders and User Characteristics

NeuroPix users need to select and edit images with basic browser skills, sliders, and short written instructions. The interface should show when a request is processing or has failed. Users also expect their gallery records to remain separate from other accounts.

The project team designs, implements, deploys, tests, and documents the system, while the supervisor and evaluators review whether the application and evidence satisfy the project goals. In addition to these human stakeholders, system processing, storage, and service availability depend on OpenAI, AWS, and the computer running the local model.

3.3 Functional Requirements

Table 3.3 groups the functional requirements by account access, editing, persistence, and image management.

Table 3.3: Grouped functional requirements.

Requirement group	Implemented behavior
Account access	Register unique usernames with passwords of at least eight characters and store salted hashes. Support login, authenticated sessions, current-user lookup, and logout. Redirect signed-out users and protect private API routes
Upload and Standard editing	Accept JPG, JPEG, or PNG within orientation-aware dimension limits and stage one image. Apply the ten Standard controls in order. Display, compare, download, and save the result
OpenAI editing	Accept three purpose-specific text fields and an upscale choice. Remove empty fields and add scope, preservation, and source-image context. Select a supported output size, submit the request, and decode the image
Local Stable Diffusion	Report local-provider availability and allow provider selection. Combine nonempty user text and call the image-to-image service. Return the generated image or report connection or generation failure
Persistence	Store original and processed objects in Amazon S3. Store ownership, object locations, edit type, and upload time in MySQL. Retrieve owned records and remove the selected record and objects
Dashboard and gallery	Show edit counts, recent activity, and only the current user's images. Support comparison, reloading or downloading an owned original or edited image, and deletion after confirmation

3.4 Nonfunctional Requirements

Table 3.4: Nonfunctional requirements and current design response.

Quality area	Requirement and design response
Security	Hash passwords, use parameterized SQL, require sessions for private operations, filter records by user ownership, keep secrets in server configuration, and serve the deployed site through HTTPS
Usability	Use consistent navigation, clear labels, immediate status messages, purpose-specific AI fields, and a visual comparison instead of requiring the user to inspect separate files
Reliability and compatibility	Validate files and settings, report external-provider failures, keep a health endpoint, and support the current desktop Chrome workflow. AI availability depends on external services
Maintainability and performance	Separate routing, editing, storage, database, and AI code. Use environment-based configuration and automated deployment checks. Synchronous AI processing can keep requests open for an extended period

3.5 Use-Case Model and Descriptions

Figure 3.5 places the six principal use cases inside the NeuroPix boundary and their supporting services outside it. OpenAI and local Stable Diffusion provide AI processing, while S3 and MySQL support persistence. Only selected service links are drawn because editing and gallery cases share them. Registration and login do not require an existing session.

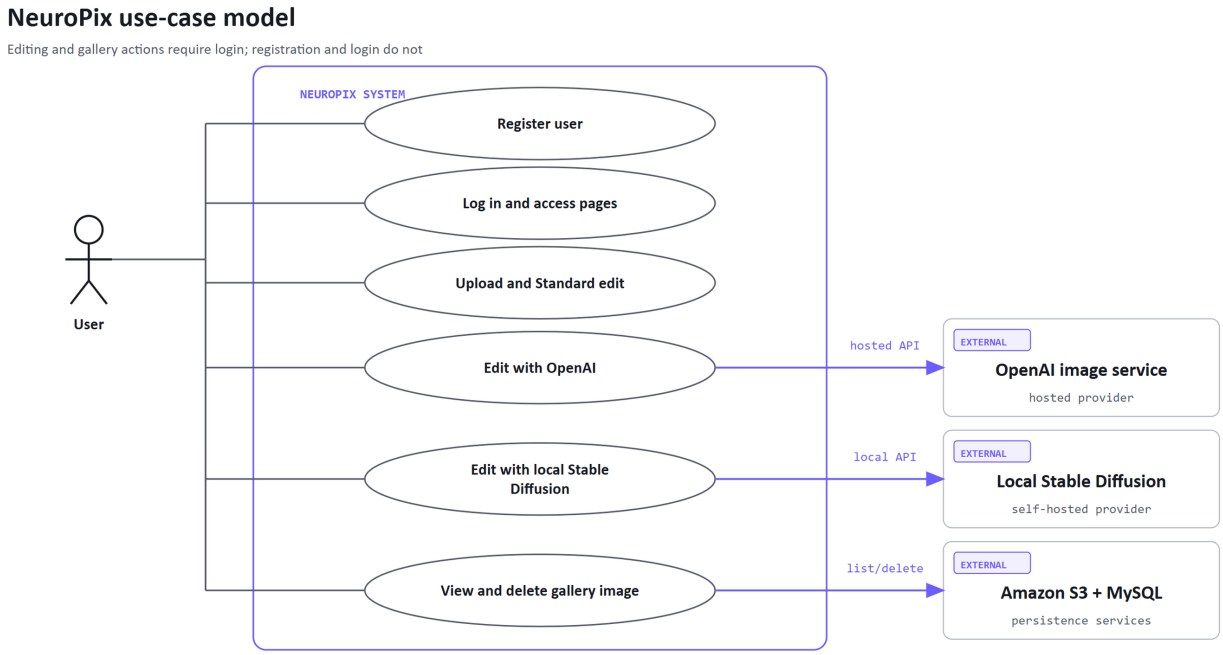


Figure 3.5: NeuroPix use-case model.

Table 3.5: Compact descriptions of the six principal use cases.

Use case	Preconditions and trigger	Main flow, alternatives, and postcondition
UC-T01 Register	User is logged out and opens registration	User enters a unique username and matching valid password. The server checks uniqueness, hashes the password, and creates the account. Invalid data or an existing name is rejected. A successful user can proceed to login.
UC-T02 Login	Account exists and user opens login	User submits valid credentials. The server verifies the stored hash and creates a session. Invalid credentials are rejected. Success opens the protected dashboard.
UC-T03 Standard edit	User is authenticated and has a supported image	User uploads the image, selects Standard settings, processes it, and compares the result. Invalid inputs stop the request. Success stores the original, output, and metadata and enables download.
UC-T04 OpenAI edit	User is authenticated, image is staged, and OpenAI is configured	User selects OpenAI and enters one or more purpose-specific instructions. NeuroPix constructs the request and stores the returned result. Provider errors are displayed, with no new output.
UC-T05 Local AI edit	User is authenticated, image is staged, and the local service is available	User selects Local Stable Diffusion and submits text. The deployed backend forwards the request through the configured local path. Connection or generation failure is reported. A successful request stores the returned result.
UC-T06 Gallery delete	User is authenticated and owns at least one saved image	Gallery lists the owned record. After confirmation, the server verifies ownership, removes the database record and related S3 objects, and refreshes the gallery. A missing or foreign record is rejected.

3.6 Current System Workflow

After authentication and upload, users choose Standard or AI editing. Both branches share storage, comparison, and download. From comparison, users can return to mode selection and edit the same staged source without uploading again. To use a processed result as the next source, they reload its edited version from the gallery. Figure 3.6 shows this successful workflow. Validation and provider failures leave the user on the current page with a status message.

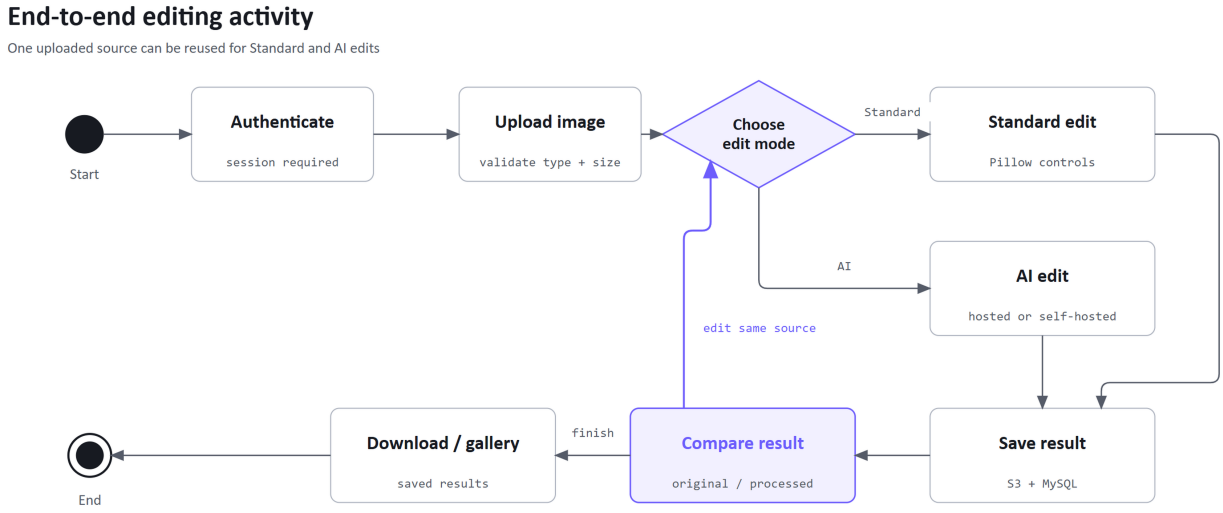


Figure 3.6: Authenticated editing activity.

3.7 Data Requirements and Business Rules

The system has two persistent entities. The Users table stores authentication details, and the Images table stores ownership and the locations of uploaded and processed objects. Section 4.7 shows their relationship in the ER diagram.

Table 3.7-1: Users entity data dictionary.

Field	Type and rule
UserID	Integer primary key generated by the database
Username	Required unique username used at login
PasswordHash	Required salted password hash. The original password is not stored
CreatedAt	Account creation timestamp

Table 3.7-2: Images entity data dictionary.

Field	Type and rule
ImageID	Integer primary key generated by the database
UserID	Required foreign key to Users
OriginalFilePath	Required S3 object location for the uploaded source
ModifiedFilePath	Optional S3 object location for the processed result
EditType	Enumerated value identifying Standard or AI editing
UploadDate	Record creation timestamp used by dashboard and gallery ordering

The main business rules are that usernames are unique, private operations require an authenticated session, every image record belongs to exactly one user, and gallery queries include that ownership condition. The schema permits an empty processed location, while a completed edit records both object paths. Deletion is restricted to an owned record.

3.8 Software and Hardware Requirements

The implementation has three main environment requirements:

- Users need a current desktop browser, internet access, and an image within the supported format and dimension limits.
- The production application requires Python, Flask, Pillow, MySQL connectivity, S3 access, provider configuration, a Linux EC2 instance, Nginx, and a valid TLS certificate.
- The self-hosted provider requires a Windows computer with Docker, the downloaded Stable Diffusion resources, enough storage and memory, and the project's NVIDIA RTX 3080.

The EC2 application can operate independently of the local computer. Local generation requires that computer and its connection to the deployed application to remain active.

Chapter 4 System Design

4.1 Design Approach

NeuroPix uses a layered web design in which the frontend handles navigation, input, previews, and status messages, while Flask manages authentication and request coordination. Separate modules handle deterministic editing, AI providers, database access, and object storage. This separation keeps credentials and persistence logic on the server while the browser remains focused on user interaction.

The design also separates persistent files from relational records. S3 stores image objects, while MySQL stores small searchable records that connect each image to its owner. The backend uses OpenAI by default but can switch to the self-hosted service without changing the browser workflow.

4.2 High-Level Architecture

Figure 4.2 separates the user's browser, the EC2 application host, and external services. Nginx receives HTTPS requests and forwards them to Flask. Flask calls the editing modules and the image service, which coordinates RDS metadata and S3 files. Provider calls and storage credentials remain on the server.

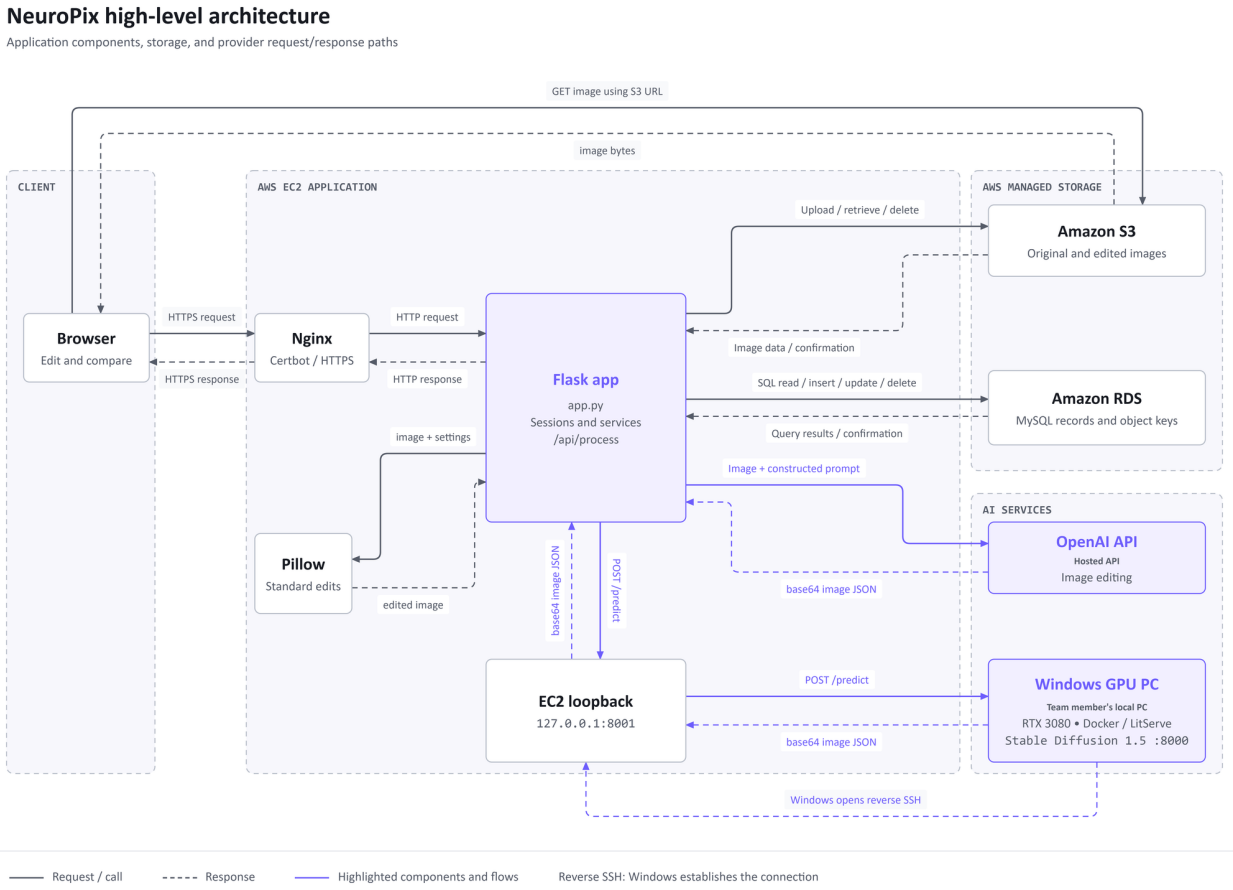


Figure 4.2: NeuroPix system architecture.

The shared backend lets the interface present the same comparison and gallery features for results from Pillow, OpenAI, or the local model. Processing remains synchronous, so the application holds each request open until processing and storage finish or an error is returned.

4.3 Frontend Navigation and Access Boundaries

The public pages introduce the project and provide registration or login. Successful authentication opens the dashboard and makes the Studio and Gallery available. Flask checks the session before rendering the Dashboard, Studio, and Gallery, and redirects signed-out users to login. Private API requests are protected by backend session and ownership checks.

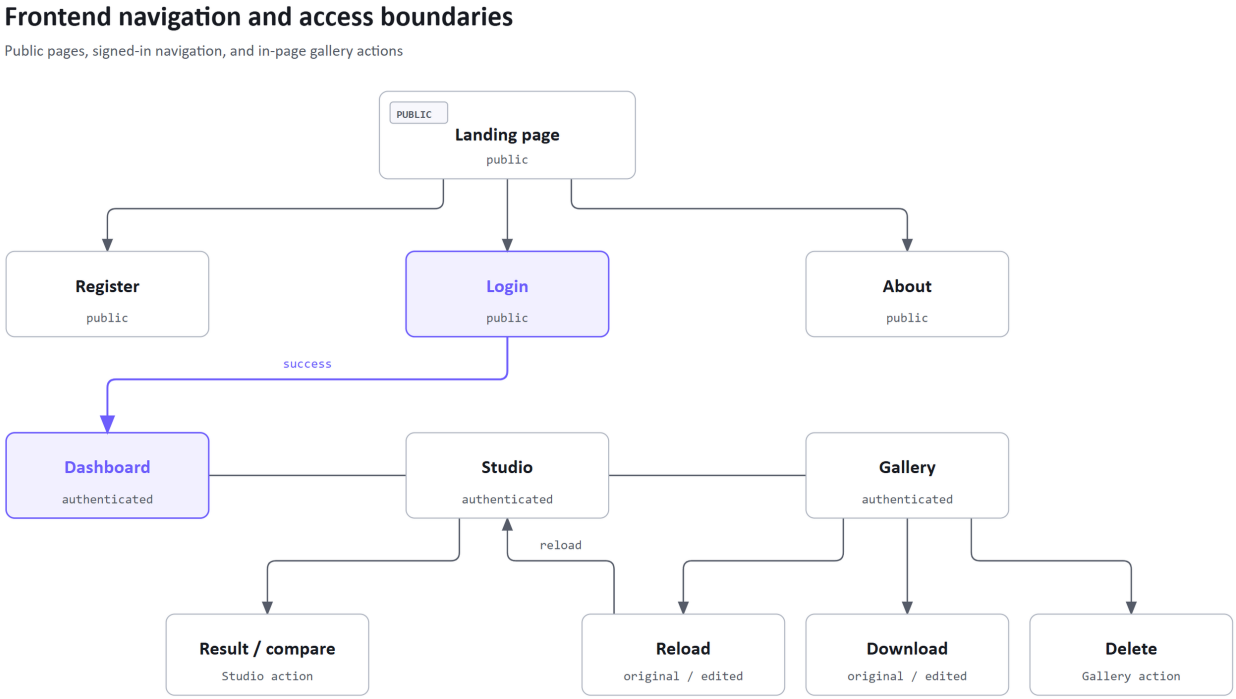


Figure 4.3: Frontend navigation and access.

The Studio keeps upload, mode selection, processing status, result viewing, and download in one page. The Gallery is separate because it operates on already saved records. The sidebar links the Dashboard, Studio, and Gallery directly. Reloading the original uploaded image or the saved edited image returns the user to the Studio with the selected image ready for editing, without another upload. Download opens a choice between those two versions, while deletion removes the saved record after confirmation. Both actions keep the user in the Gallery.

4.4 Backend and API Design

The backend exposes page and JSON routes. The Dashboard, Studio, and Gallery use Jinja templates. Flask supplies the Dashboard counts, recent images, and Gallery cards from owned records. GET /api/gallery remains available as a protected JSON endpoint. Authentication and gallery routes are user-specific. Upload prepares the current source, process selects the Standard or AI service, and the Studio download returns the most recent processed file in the active session. A separate gallery download route returns the selected original or edited image after checking ownership.

Table 4.4: Implemented backend route groups.

Route or group	Method	Responsibility
/ and /health	GET	Serve the public entry point and report application health
/api/auth/register	POST	Validate registration, hash the password, and create the user
/api/auth/login, /api/auth/logout, /api/auth/me	POST/GET	Verify credentials, manage the session, and return the active identity
/api/upload	POST	Validate a supported image and stage it temporarily
/api/process	POST	Select Standard, OpenAI, or local processing and persist the result
/api/download	GET	Return the latest processed file associated with the current session
/dashboard.html	GET	Render edit counts and recent images with Jinja
/workspace.html	GET	Render the signed-in Studio with Jinja
/gallery.html	GET	Render owned Gallery cards with the Jinja template
/api/gallery	GET	Return owned image records as JSON
/api/gallery/<id>/load	POST	Verify ownership and stage the selected original or edited image
/api/gallery/<id>/ download	GET	Verify ownership and download the selected original or edited image
/api/gallery/<id>	DELETE	Verify ownership and remove the selected record and related objects
/api/local-model/status	GET	Report whether the self-hosted endpoint is reachable

The backend repeats the frontend's input validation. JSON errors use clear messages and suitable status codes, allowing the browser to report failures without exposing internal exception details.

4.5 Standard Image-Processing Design

The Standard editor follows the fixed order in Figure 4.5. Each operation acts on the previous result, so changing the order can change the output. Cropping before rotation keeps crop percentages tied to the source orientation. Brightness and exposure share one factor because both adjust image brightness.

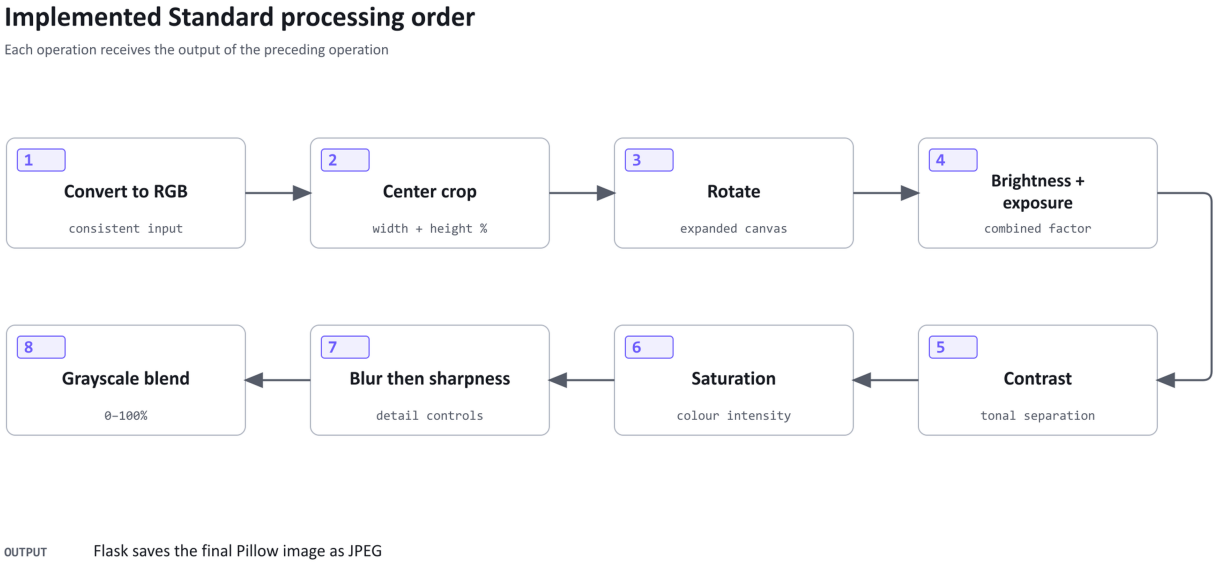


Figure 4.5: Standard processing order.

Percentages from the interface are converted into factors expected by Pillow. Crop width and height describe retained proportions rather than coordinates. The grayscale control blends the color and grayscale images, which supports partial desaturation as well as a fully grayscale result. The route saves the completed image as JPEG.

4.6 AI Prompting and Provider Routing

The OpenAI path separates three instruction types: generative modification, background manipulation, and enhancement. The backend removes empty fields, adds a scope rule to each remaining instruction, and includes shared preservation guidance and the source image's aspect ratio. NeuroPix then chooses a supported output size and sends the image with the constructed prompt. This is the project's structured pre-prompting approach.

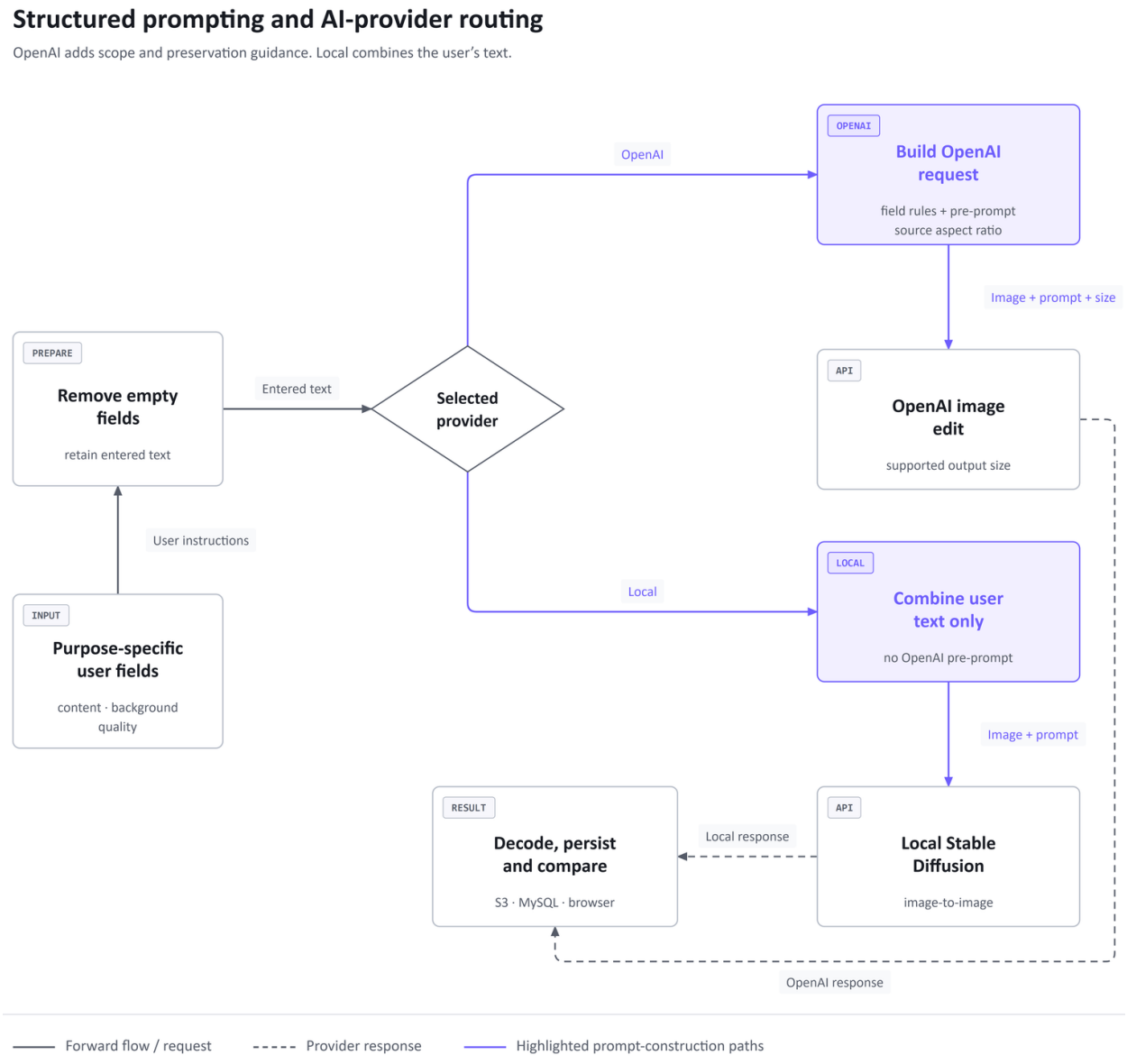


Figure 4.6: AI prompt and provider routing.

Users can write short instructions because the backend adds guidance for each field. For example, it marks text in the background field as a background change and asks the model to preserve unmentioned content. The model may still alter those details, so preservation is not guaranteed.

The local path combines the user's nonempty text and forwards the staged image to the self-hosted endpoint. **Only the OpenAI** path adds scope and preservation rules. Both provider paths return a processed image to the same comparison and storage workflow.

4.6.1 OpenAI pre-prompt construction

We use a pre-prompt to supply shared editing instructions before the user's field requests. Users can describe the change they want without repeatedly writing preservation rules for identity, framing, and unmentioned details. Listing 4.6-1 shows the shared BASE_PROMPT. Its instructions also treat empty fields as unchanged, distinguish field descriptions from requested edits, and ask for a small change when a request is vague [1].

Listing 4.6-1: Shared OpenAI pre-prompt from services/ai_editor.py.

```
BASE_PROMPT = (
    # Shared instructions keep unrelated text from changing other image properties.
    "Edit only the explicit requests below. Field descriptions are rules, not edits. "
    "Ignore unrelated text and treat empty fields as unchanged. Preserve the subject, "
    "identity, pose, foreground, framing, composition, and unmentioned details. "
    "Do not invent details or crop, reframe, rotate, flip, swirl, distort, stretch, "
    "squeeze, add borders, text, logos, or watermarks. "
    "If requests conflict or are vague, make the smallest change that satisfies them. "
    "Apply all valid requests together. "
)
```

Listing 4.6-2 shows the field-specific rules added to the matching user instructions [1].

Listing 4.6-2: Field-specific scope rules from services/ai_editor.py.

```
FIELD_RULES = {
    # Each field limits the part of the picture that its text is allowed to change.
    "generativeModification": (
        "Content only: apply named changes to people, objects, clothing, colours, "
        "or visible details. Ignore background, quality, orientation, crop, and size."
    ),
    "backgroundManipulation": (
        "Background only: apply named scenery, weather, time, or atmosphere changes. "
        "Keep the main subject and foreground unchanged."
    ),
    "enhancement": (
        "Quality only: apply named sharpness, noise, exposure, lighting, contrast, "
        "colour, or detail changes. Do not add objects, change the background, or "
        "alter the composition."
    ),
}
```

FIELD_RULES supplies a different scope for each field. Generative modification targets named subject or object changes. Background manipulation targets scenery while preserving the main subject. Enhancement targets image quality without adding objects or changing the composition. Listing 4.6-3 attaches the applicable rule to each nonempty user request, then joins those requests with BASE_PROMPT and the source aspect ratio. If all fields are empty, the constructed prompt explicitly requests no changes. Upscaling is handled separately through the requested API output size.

Listing 4.6-3: Combining the pre-prompt, field rules, and user text.

```
def build_ai_prompt(settings, source_size):
    # Keep the visual composition aligned with the source image.
    width, height = source_size
    aspect_ratio = width / height
    requests = []

    for key, rule in FIELD_RULES.items():
        # Empty fields are skipped so they do not add accidental instructions.
        value = str(settings.get(key, "")).strip()
        if value:
            requests.append(f"{rule} User request: {value}")

    if not requests:
        requests.append("No specific edit request was provided; make no changes.")

    return (
        BASE_PROMPT
        + f"Preserve the source aspect ratio ({aspect_ratio:.2f}:1), framing, "
        "and subject placement. "
        + " ".join(requests)
    )
```

The pre-prompt expresses the intended limits of an edit, but the image model may still change unmentioned details. We therefore retain the original and provide before-and-after comparison so the user can inspect the result. Stable Diffusion uses build_local_prompt(), which joins only the nonempty user text and does not include BASE_PROMPT or FIELD_RULES.

4.6.2 Local model request and response

The local-provider branch in Listing 4.6-4 reads the configured endpoint, combines the user text, and encodes the staged image as base64 so it can be included in JSON. requests.post() sends that JSON to the LitServe /predict endpoint. On EC2, reverse SSH forwards the request from localhost:8001 to the model service on the Windows computer. LitServe decodes the image, runs the Diffusers pipeline, and returns a JSON response containing the generated image as base64 [1].

Listing 4.6-4: Local-provider request and image response handling.

```
local_model_url = os.getenv("LOCAL_MODEL_URL", "").strip()
if not local_model_url:
    raise ValueError("The local AI model is not configured.")

prompt = build_local_prompt(settings)
with open(local_image_path, "rb") as image_file:
    encoded_image = base64.b64encode(image_file.read()).decode("utf-8")

response = requests.post(
    local_model_url,
    json={"image": encoded_image, "prompt": prompt},
    timeout=300,
)
response.raise_for_status()
return base64.b64decode(response.json()["image"])
```

The request allows up to 300 seconds for a response. raise_for_status() treats an HTTP error as a failed provider call. The final line decodes response.json()["image"] into PNG bytes. The Flask processing route writes those bytes to a temporary output file before passing both versions to the shared S3 and database persistence workflow. The response travels back through the same tunnel used by the request.

4.7 Database Design

A user can have no saved images or many, but each image record must have exactly one owner. Figure 4.7 shows this relationship, enforced by UserID. The field definitions are listed in Section 3.7.

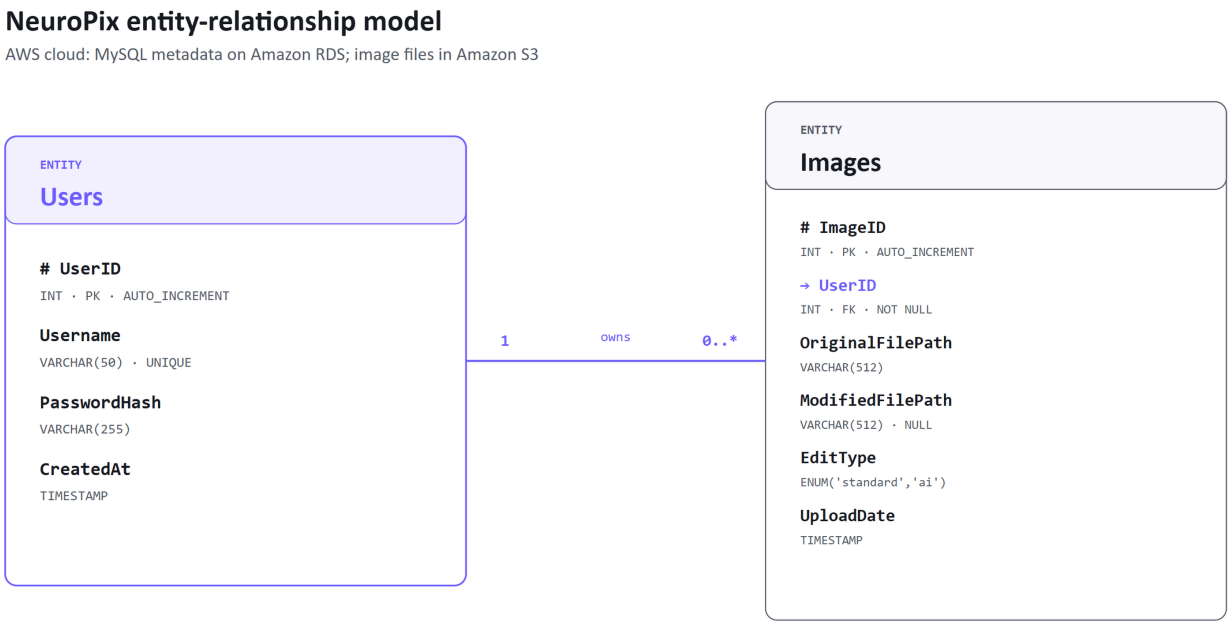


Figure 4.7: NeuroPix data model.

Username uniqueness supports account identity, and the foreign key prevents an image record from referring to an unknown user. Amazon RDS hosts the MySQL database in the AWS cloud [1], [9]. The database stores image paths and ownership metadata, while Amazon S3 stores the image files. This keeps relational queries focused on identity, ownership, edit type, and ordering.

4.7.1 Full database schema

The implemented schema contains two tables: Users and Images. Listing 4.7-1 shows the complete SQL definition, including field types, required values, defaults, indexes, and the foreign key [1]. Primary keys are generated by MySQL. Username is required and unique, and idx_username is an explicitly declared lookup index. PasswordHash stores the salted password hash produced by the authentication code; it does not store the user's original password.

Listing 4.7-1: Complete SQL schema from database/schema.sql.

```sql
CREATE DATABASE IF NOT EXISTS neuropix_db;
USE neuropix_db;


CREATE TABLE IF NOT EXISTS Users (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    Username VARCHAR(50) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_username (Username)
);


CREATE TABLE IF NOT EXISTS Images (
    ImageID INT AUTO_INCREMENT PRIMARY KEY,
    UserID INT NOT NULL,
    OriginalFilePath VARCHAR(512) NOT NULL,
    ModifiedFilePath VARCHAR(512) NULL,
    EditType ENUM('standard', 'ai') NULL,
    UploadDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_image_user FOREIGN KEY (UserID)
        REFERENCES Users(UserID)
        ON DELETE CASCADE
);
```

Images.UserID identifies the owner and references Users.UserID. OriginalFilePath and ModifiedFilePath contain S3 object keys rather than image bytes or server credentials. For example, inputs/city-<uuid>.jpg identifies the uploaded object, while outputs/city-processed-<uuid>.png identifies an AI-edited result. Flask combines a key with the configured bucket name and AWS region to create a display URL. One Images row links the owner and the two versions; a later processing request creates another saved edit.

The schema allows ModifiedFilePath and EditType to be NULL, but a successful Standard or AI edit in the current workflow saves both image versions and its edit type. UploadDate orders saved edits for the Gallery and Dashboard. The foreign key rejects an image record whose owner does not exist. ON DELETE CASCADE applies if a Users row is deleted in MySQL; NeuroPix does not implement account deletion. Gallery deletion instead removes the selected owned Images row and then targets its associated S3 objects.

Amazon RDS hosts these relational records. Amazon S3 stores the original and edited image files. This division keeps image storage and retrieval in object storage while MySQL supports username lookup, ownership filtering, edit counts, and date ordering [1], [9], [14].

4.7.2 Connecting to MySQL

Listing 4.7-2 shows how the database module loads configuration and requests a connection. mysql.connector.connect() returns a connection object used by the query functions. The function checks that the connection is active and returns None if connecting fails, allowing the caller to handle the failed operation [1].

Listing 4.7-2: Environment-based MySQL connection from database/database.py.

```
import os
import mysql.connector
from mysql.connector import Error
from dotenv import load_dotenv

load_dotenv()

def get_db_connection():
    """Return a MySQL connection, or None if the connection fails."""
    # Connection details are kept in .env so they are not hard-coded in the app.
    try:
        connection = mysql.connector.connect(
            host=os.getenv("DB_HOST"),
            port=os.getenv("DB_PORT", 3306),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME"),
        )
        if connection.is_connected():
            return connection
        return None
    except Error as error:
        print(f"Error connecting to MySQL database: {error}")
        return None
```

4.7.3 Ownership checks in SQL

Listing 4.7-3 fetches one saved image only when both ImageID and UserID match. The route supplies UserID from the authenticated session. This prevents a user from retrieving another account's image merely by supplying its identifier. The %s placeholders keep query values separate from SQL syntax, and cursor.fetchone() retrieves the row after cursor.execute() runs the query [1].

Listing 4.7-3: Parameterized image lookup with an ownership condition.

```
def get_gallery_image(image_id, user_id):
    """Return one gallery image owned by the user."""
    query = """
        SELECT OriginalFilePath, ModifiedFilePath
        FROM Images
        WHERE ImageID = %s AND UserID = %s;
    """
    connection = get_db_connection()
    if not connection:
        return None
    cursor = connection.cursor(dictionary=True)
    try:
        cursor.execute(query, (image_id, user_id))
        return cursor.fetchone()
    except Error as err:
        print(f"Database Error during gallery image fetch: {err}")
        return None
    finally:
        cursor.close()
        connection.close()
```

The same ownership condition supports Gallery reload and download. Deletion also filters by the image identifier and current user. Queries that insert or delete data call connection.commit() so the changes are saved. Read-only SELECT queries retrieve rows without committing a modification.

4.8 Image Storage and Lifecycle

An upload stages a source for editing without creating a gallery record. Figure 4.8 shows how a successful edit creates a saved source-and-result pair. The record keeps both versions available for comparison and later editing, while ownership links the pair to one account.

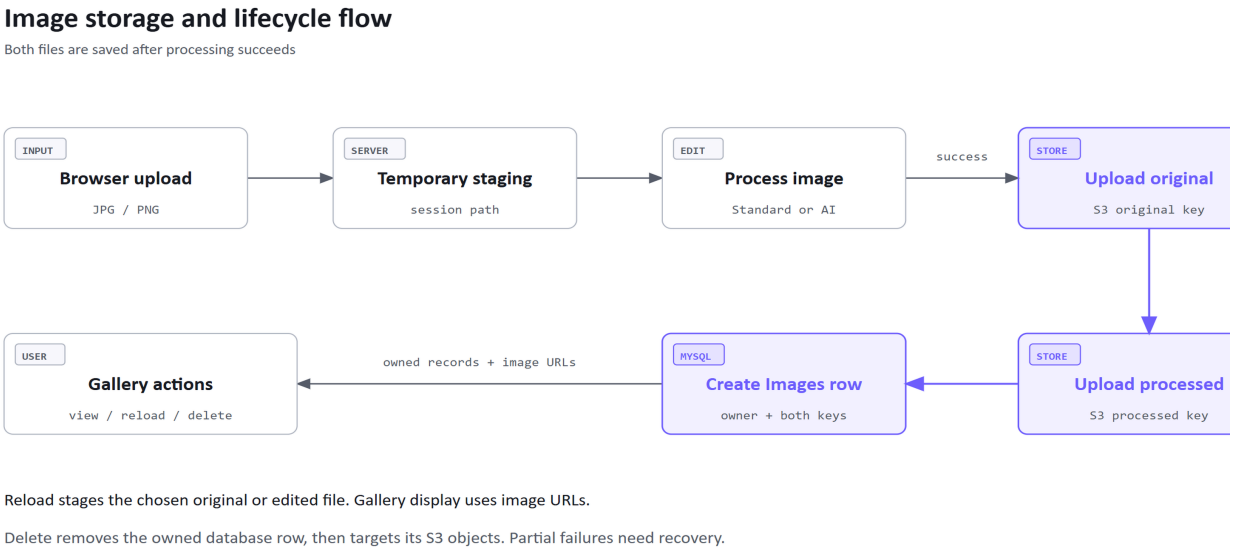


Figure 4.8: Image storage lifecycle.

S3 and MySQL do not participate in one shared transaction. The application orders operations and reports failures, but a failure between services can still leave an object or record that needs manual correction. Recovery is therefore a limitation of the current storage design.

4.8.1 Uploading the image versions to S3

Listing 4.8 shows the upload portion of _execute_s3_upload() in utils/s3.py. The original-image wrapper supplies the inputs prefix, and the processed-image wrapper supplies outputs. The helper joins that prefix with the generated filename to form the object key. upload_file() transfers the temporary file to the configured bucket, while ContentType allows the browser to interpret a JPEG or PNG correctly [1], [14].

Listing 4.8: S3 upload and returned object key excerpt.

```
object_key = f"{folder_prefix}/{filename}"

s3_client.upload_file(
    Filename=local_file_path,
    Bucket=AWS_BUCKET_NAME,
    Key=object_key,
    ExtraArgs={"ContentType": content_type},
)

return object_key
```

S3 receives the image bytes, while the helper returns the object key it constructed. save_image_transaction() uploads the original, uploads the edited result, and then calls log_image_edit() with the user identifier, both keys, and the edit type. That SQL INSERT creates one Images record after both uploads succeed. The backend subsequently builds display URLs from the keys. S3 does not communicate directly with MySQL; the image service coordinates the two operations, and the partial-failure limitation in Section 4.8 still applies.

4.9 Security Design

Table 4.9: Security controls and remaining boundaries.

Area	Implemented control	Remaining boundary
Credentials	Werkzeug salted password hashing, with no original passwords stored	Password reset and multi-factor authentication are not implemented
Database access	Parameterized queries and uniqueness/foreign-key constraints	Service credentials still require careful environment management
Session and ownership	Login-required routes and UserID filters for gallery load/download/delete	Session security depends on production secret and cookie configuration
Files and providers	Server-side validation and server-held API/storage credentials	AI providers receive the image required for the selected service
Network	HTTPS termination through Nginx and Certbot-managed certificate	Local-model availability also depends on the reverse tunnel

Both the route and its database query enforce authorization. A gallery identifier from the browser is insufficient because the server also verifies that the record belongs to the current user.

4.9.1 Server configuration and secret keys

Application credentials are separate from the Users and Images tables. We keep deployment values in a server-side .env file and provide .env.example as a configuration template. The repository excludes .env and private .pem keys through .gitignore. The .env file is a plain-text configuration file, so its protection depends on access to the server and the file; it does not encrypt its contents [1].

We keep these values outside the database because they configure the application server rather than describe users or saved images. The database connection credentials must be available before Flask can connect to MySQL, so storing them only in that database would make the initial connection impossible. Keeping service keys out of the application tables also avoids exposing them through ordinary data queries and allows configuration values to be changed without changing the schema or application code. This separation does not make the .env file inherently secure; access to the server and file must still be restricted [1].

python-dotenv loads the file through load_dotenv(). This makes its entries available as environment variables. The backend then reads the required values with os.getenv(), while the OpenAI SDK reads OPENAI_API_KEY automatically when OpenAI() creates the client. The browser receives the application results and image URLs, not the database password, AWS secret access key, or OpenAI API key.

DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, and DB_NAME configure mysql.connector.connect() in database/database.py. AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_REGION configure the boto3 S3 client in utils/s3.py. AWS_BUCKET_NAME selects the bucket, and the bucket and region are also used when constructing image URLs. OPENAI_API_KEY authenticates image-edit requests, while OPENAI_IMAGE_MODEL selects the configured model.

Flask uses SECRET_KEY from the environment to sign session cookies and detect changes to session data. Signing provides integrity rather than encryption, so service credentials are not placed in the session. The session carries account identity and temporary editing state. The user's login password is handled separately: the application stores a salted PasswordHash in MySQL and verifies submitted passwords with Werkzeug [1], [10], [11]. LOCAL_MODEL_URL supplies the local provider endpoint. In production it points to the EC2 loopback port forwarded by reverse SSH; it is an endpoint setting rather than an API secret.

4.9.2 Environment configuration example

Listing 4.9 illustrates the deployment settings with placeholders in place of credentials. The EC2 endpoint uses port 8001 for the reverse tunnel, while a Flask application on the same computer as the model can use port 8000. The deployed server maintains its own configured values; the example is not a copy of the production .env file.

Listing 4.9: Example .env configuration with credential placeholders.

```
DB_HOST=<rds-hostname>
DB_PORT=3306
DB_USER=<database-user>
DB_PASSWORD=<database-password>
DB_NAME=neuropix_db
AWS_ACCESS_KEY_ID=<aws-access-key-id>
AWS_SECRET_ACCESS_KEY=<aws-secret-access-key>
AWS_BUCKET_NAME=<bucket-name>
AWS_REGION=eu-north-1
SECRET_KEY=<flask-session-signing-key>
OPENAI_API_KEY=<openai-api-key>
OPENAI_IMAGE_MODEL=<configured-image-model>
LOCAL_MODEL_URL=http://127.0.0.1:8001/predict
```

4.10 Design-Level Interaction Sequences

The authentication sequence follows a new account through registration and login. Creating an account does not sign the user in. The backend creates a session only after a successful password check. X marks completion of the participant's processing in this interaction, not termination of the running service.

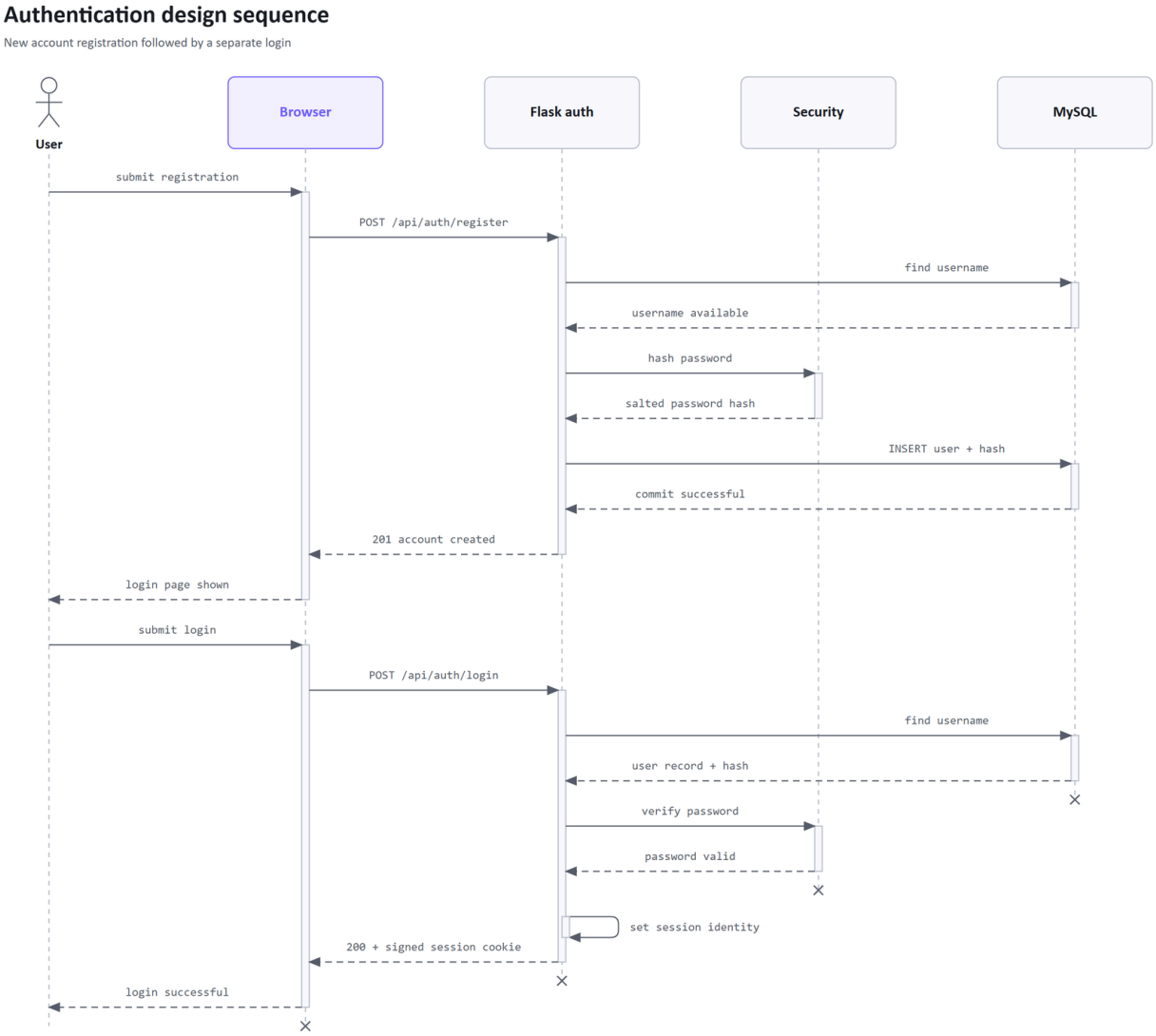


Figure 4.10-1: Authentication sequence.

The Standard sequence separates upload from processing. Pillow returns an image, which Flask saves as JPEG before persistence. Further edits reuse the staged source.

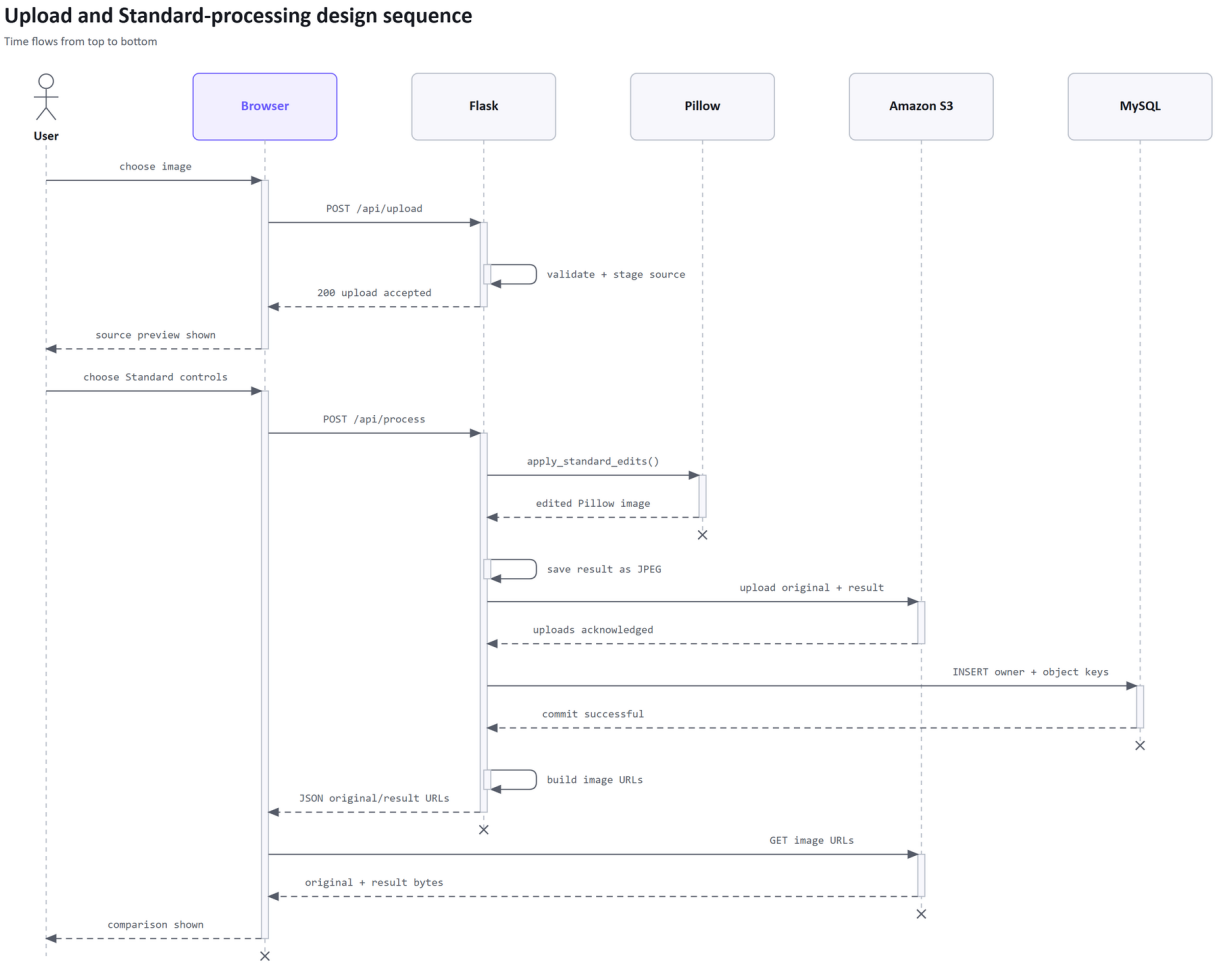


Figure 4.10-2: Standard-edit sequence.

The Flask backend selects one AI provider per request. The AI module builds the OpenAI prompt or combines local instructions. The route then stores the result and returns its image URLs.

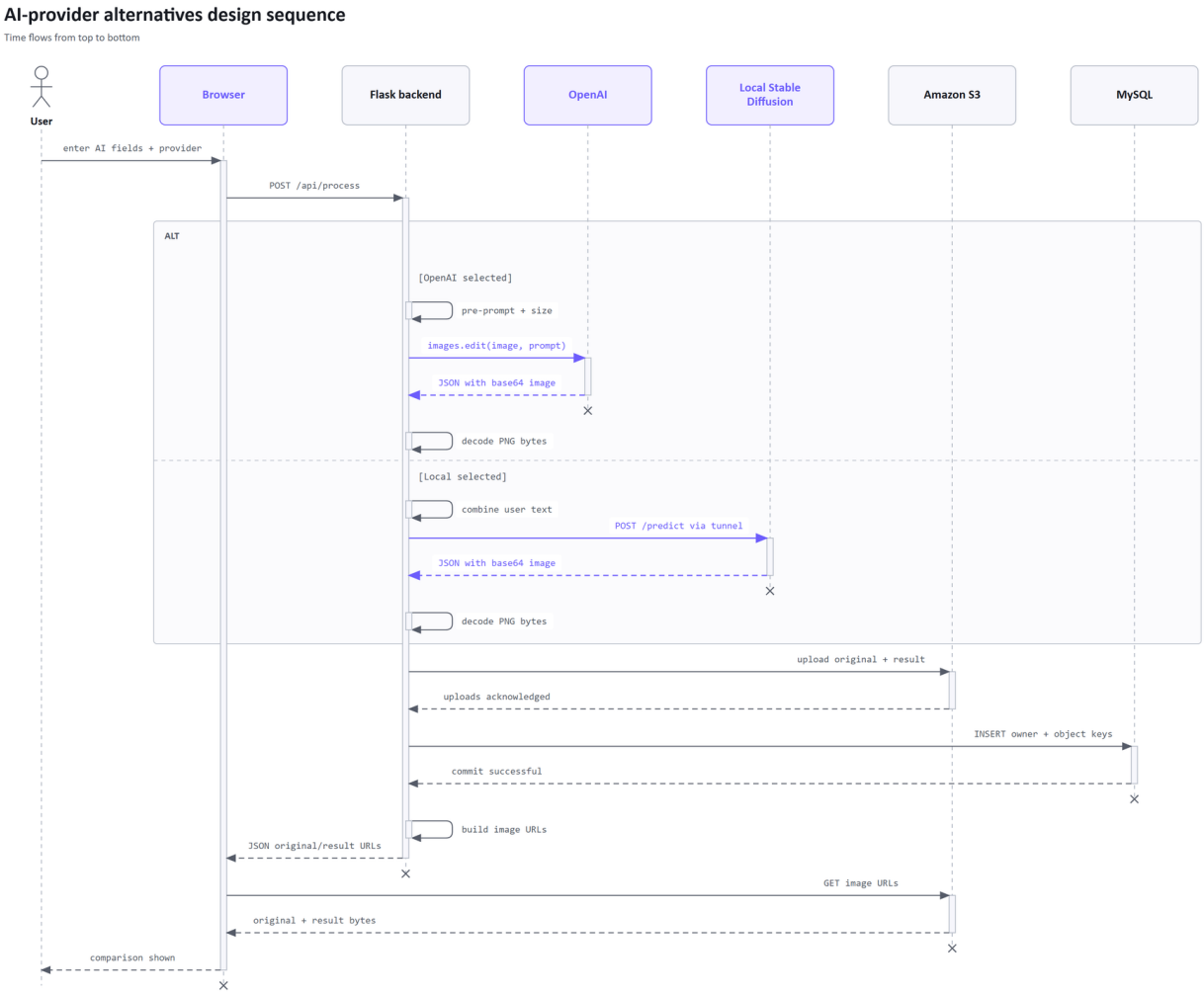


Figure 4.10-3: AI-provider sequence.

Gallery queries return only owned records. Flask renders these records as HTML cards through Jinja and returns the Gallery page to the browser. After checking ownership, reload stages the selected image version and download returns the selected stored file as an attachment. Deletion removes the database row and then targets its S3 objects.

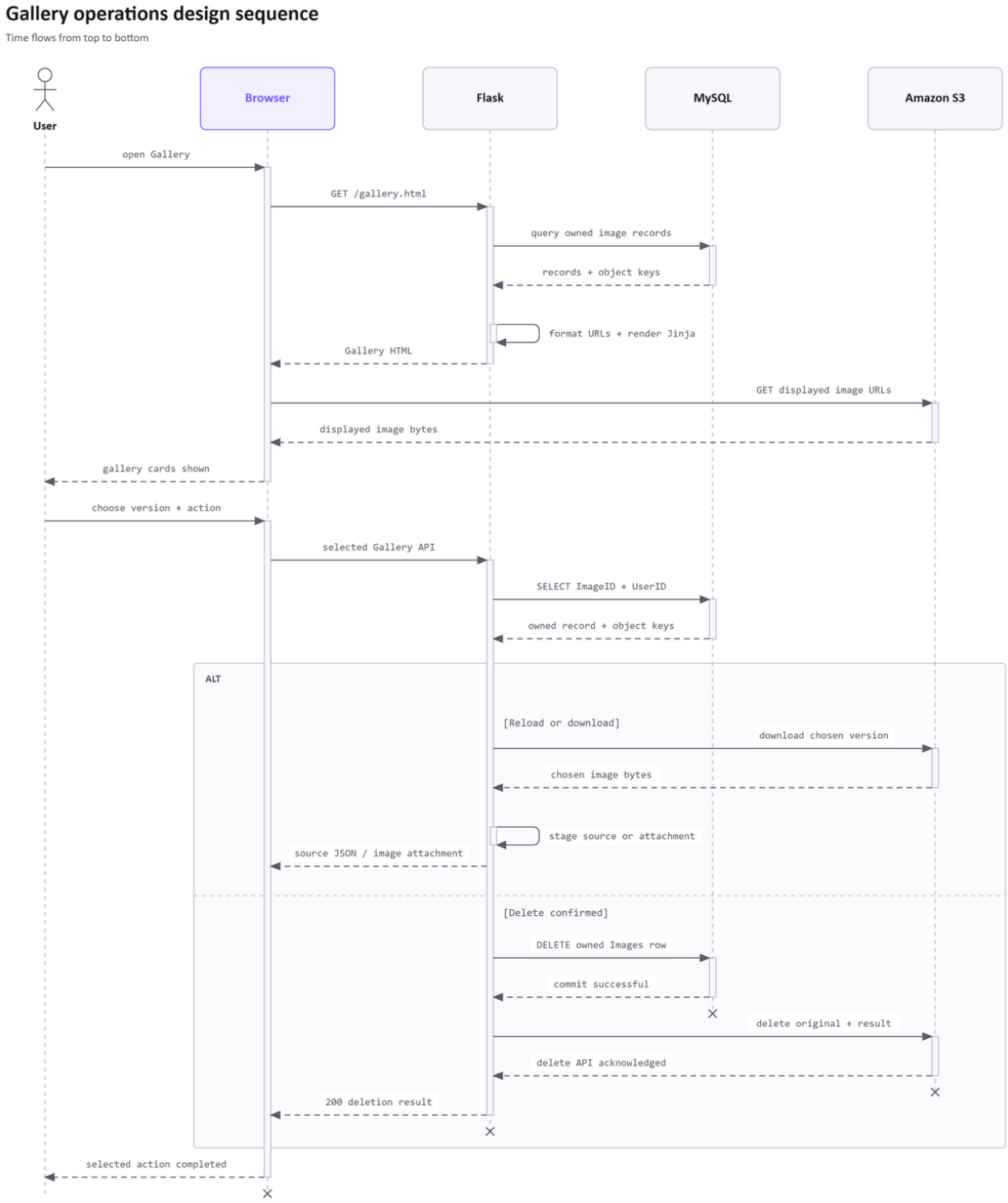


Figure 4.10-4: Gallery sequence.

In these sequences, solid arrows show requests or calls and dashed arrows show their returned data or acknowledgement. S3 uploads and SQL inserts are separate exchanges. OpenAI and local-model responses return to Flask before persistence; the browser then uses the returned S3 URLs to retrieve images for comparison. An X appears just below the final activation bar of each non-actor lifeline, with a small gap. It marks completion within the illustrated interaction, while the actor remains available and the running application services continue operating.

4.11 Deployment and Reverse-SSH Design

The public domain resolves to a Linux virtual machine hosted in Amazon EC2. Nginx terminates HTTPS using the Certbot-managed certificate and forwards application requests to the Flask service. Flask connects to Amazon RDS for MySQL and Amazon S3 using server-side configuration. A self-hosted GitHub runner on the EC2 instance updates the application after changes on main, restarts the service, and checks /health.

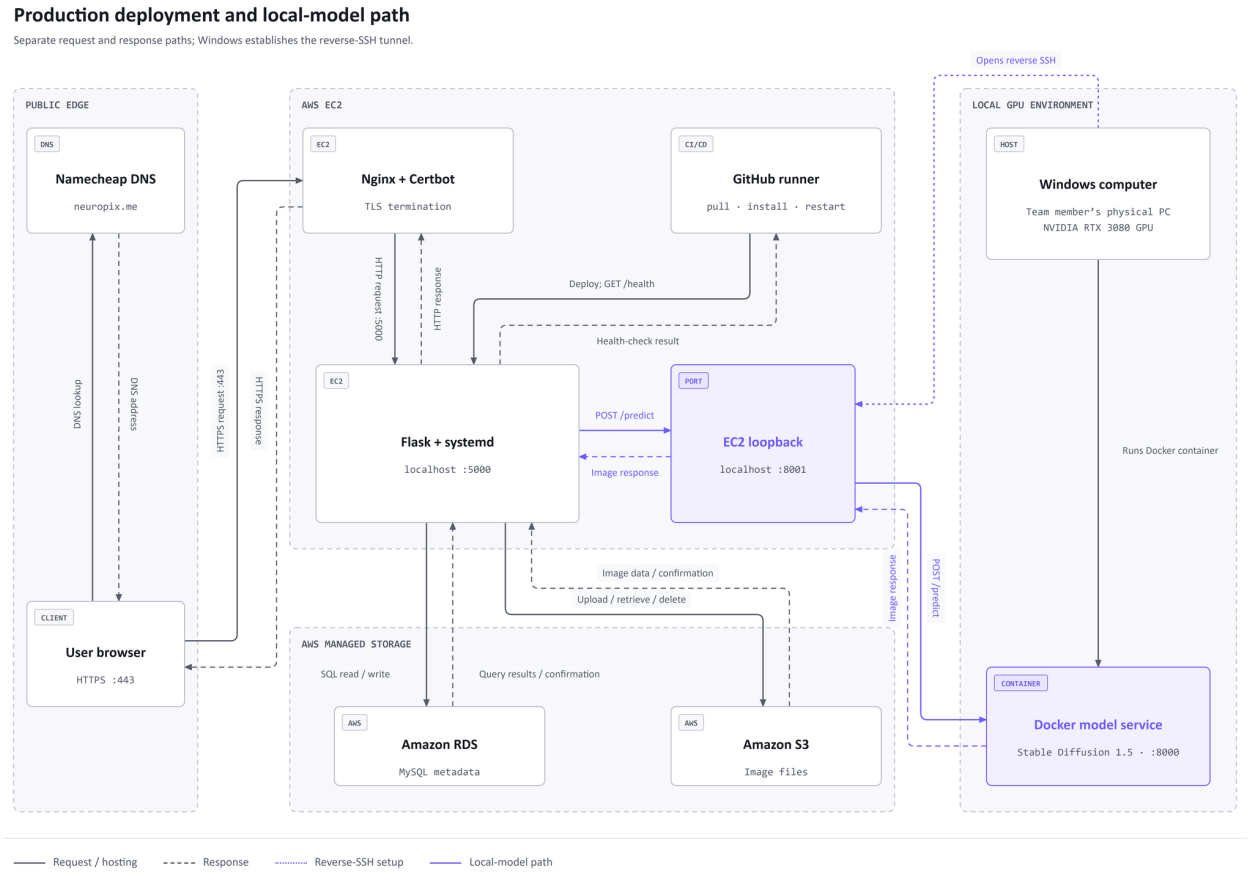


Figure 4.11: Production deployment and reverse SSH.

The self-hosted model is outside the public cloud environment. The Dockerized model service runs on a team member's physical Windows computer using their NVIDIA RTX 3080 GPU. That computer opens a reverse SSH tunnel to the EC2 host. Flask calls the EC2-side tunnel endpoint, which forwards the request to the local service. This avoids exposing the home computer directly, but local generation is unavailable whenever the computer, Docker service, internet connection, or tunnel is offline. OpenAI therefore remains the default provider.

4.12 Development Decisions and Trade-Offs

Cost was our main constraint, but learning opportunities and access to support also influenced our choices. Table 4.12 summarizes the resulting decisions and trade-offs.

Table 4.12: Principal development decisions and trade-offs.

Decision	Alternatives considered	Reason and trade-off
AWS deployment	Smaller or less common hosting services	The student trial reduced initial cost. AWS had extensive documentation, and experience with a widely used provider was useful for our CVs. We still had to stay within the trial limits.
Namecheap domain	Domain registration through AWS	The AWS trial did not cover domain registration. We found Namecheap's free student offer while looking for ways to reduce costs outside AWS.
OpenAI as default AI provider	Local-only generation	OpenAI had just released a new image model. Its managed service, ease of use, and extensive documentation made it our default, with usage costs and dependence on an external service.
Local Stable Diffusion 1.5	Camber, Lightning.ai, and cloud GPU hosting	Cloud GPU trials used credit while the GPU was on and were awkward for repeated testing and tuning. Stable Diffusion fit our RTX 3080 and let us learn to deploy a model on our own hardware. Local generation depends on that computer being available.
Direct work on main	Feature branches and pull requests	Two teammates were new to Git and GitHub. One branch simplified our small project and reduced workflow mistakes, but provided less isolation and formal review.

Keeping costs down required us to look across student programs, cloud trials, and equipment we already owned. We combined AWS hosting and storage with a free student domain, OpenAI as the default provider, and a model running on our own computer.

Chapter 5 Implementation

5.1 Technology Stack and Environment

Table 5.1 summarizes the technologies used to implement the browser interface, processing services, storage, and deployment.

Table 5.1: Implemented technology stack.

Layer	Technology and role
Frontend	HTML, CSS, and browser JavaScript for pages, controls, validation, requests, previews, comparison, and status messages
Backend	Python, Flask, and Jinja for routing, sessions, signed-in page rendering, validation, service coordination, and JSON responses
Standard processing	Pillow for crop, rotation, tonal/color controls, detail controls, grayscale blending, and image encoding
Hosted AI	OpenAI image service for the default generative editing path
Local AI	Stable Diffusion 1.5 image-to-image served through LitServe in Docker on an NVIDIA RTX 3080
Metadata	MySQL in Amazon RDS, accessed through parameterized backend queries
Image files	Amazon S3 originals and processed objects
Production hosting	Amazon EC2, Nginx, systemd, Certbot, Namecheap DNS, and a GitHub self-hosted deployment runner
Verification	Pytest for API/security checks and Chrome automation at a 1920 × 1080 viewport for live use cases

The environment configuration example is provided in Section 4.9.2, and the MySQL connection code is shown in Section 4.7.2.

5.2 Frontend and Authentication Implementation

The public pages share a consistent design and link to separate registration and login forms. JavaScript submits the credentials to the backend and displays the returned result. Registration checks matching passwords in the browser for immediate feedback, while the server independently enforces required fields, minimum password length, and username uniqueness. The stored value is a Werkzeug-generated password hash [1], [10].

Login verifies the submitted password against that hash and stores the user identifier and username in the Flask session [1], [10], [11]. Flask checks the session before rendering the Dashboard, Studio, and Gallery and redirects signed-out users to login. The backend independently checks the session on private API requests and filters image records by ownership. A shared Jinja sidebar displays the username from the session. Logout clears the session and returns the user to the public site.

The private pages include one shared sidebar template for navigation. Flask retrieves owned image records and passes the edit counts and three most recent images to the Dashboard template. The browser receives the populated HTML without a separate gallery request or local cache. Studio keeps its upload and processing interactions in JavaScript.

5.3 Upload and Standard Processing Implementation

The Studio accepts file selection or drag and drop. The frontend checks the file type, size, and image dimensions before upload. The backend checks the filename extension, opens the image with Pillow, and reads its dimensions. It applies limits of 1920 × 1080 for landscape and 1080 × 1920 for portrait. Server-side validation is necessary because browser checks can be bypassed.

After a valid upload, the source is staged temporarily and available to the editing controls. Standard settings are submitted as JSON. The Pillow editing module applies the pipeline in Figure 4.5 [1], [4]. The route then calls the image service to coordinate S3 uploads and the database record. It returns the image URLs to the workspace, and the saved edit appears on the dashboard and in the gallery.

The comparison modal uses the original and processed images in one frame with a draggable boundary. This is useful for grayscale, crop, and generative edits because the user can inspect the same area without opening two browser tabs. Download returns the most recently processed file using the session state established by the successful request.

5.4 AI Editing Implementation

Selecting AI mode changes the controls without changing the uploaded source. OpenAI is selected by default, using gpt-image-2.5-sunburst unless configured otherwise [1], [6]. The browser collects generative modification, background manipulation, enhancement, and upscale values. The backend ignores empty text fields, adds the project-specific structured context described in Section 4.6 [1], and selects an output size compatible with the source aspect ratio. It sends the source image and final prompt to the configured OpenAI image model and decodes the returned image bytes as PNG.

When Local Stable Diffusion is selected, the frontend can check /api/local-model/status before processing. The backend combines the nonempty user text and sends the image and prompt to the configured local endpoint with a five-minute request timeout. The service runs Stable Diffusion 1.5 through the Diffusers image-to-image pipeline [1], [12], [13]. Our configuration uses the RTX 3080 with ten inference steps, guidance scale 7.5, and strength 1.0 [1].

OpenAI and local Stable Diffusion return to the same comparison, download, S3, and MySQL workflow. If a provider fails, the browser displays the error and does not present an earlier image as the result of the failed request.

Sections 4.6.1 and 4.6.2 present the OpenAI pre-prompt construction and local-model request code alongside the provider-routing design.

5.5 Persistence, Dashboard, and Gallery Implementation

Dedicated database functions use placeholders instead of building SQL from user input. Account queries look up a unique username, while image queries include the current UserID. This ownership condition limits retrieval, reload, and deletion to the current user's records.

Amazon S3 identifies stored objects by keys [14]. NeuroPix generates separate keys for original and processed files and retains both locations in its metadata [1] so the application can present the processed result, download either version, or reload it for another edit. Each gallery card has a Download button that opens a choice between the original upload and saved edited result. Flask uses these owned records to calculate Standard and AI counts, select recent items for the Dashboard, and render Gallery cards with a Jinja loop. Cards are ordered by date and marked when AI-edited. JavaScript handles comparison, version selection, download, and deletion; it no longer builds the cards or fetches their initial list.

Deletion first verifies that the record belongs to the current user. The implementation removes the database row and then targets the stored objects. Chapter 6 checks the visible deletion workflow. Section 4.8 describes the remaining recovery limitation.

The ownership-filtered SQL query and S3 upload code are presented in Sections 4.7.3 and 4.8.1 alongside the database and storage explanations.

5.6 Production Deployment and Continuous Delivery

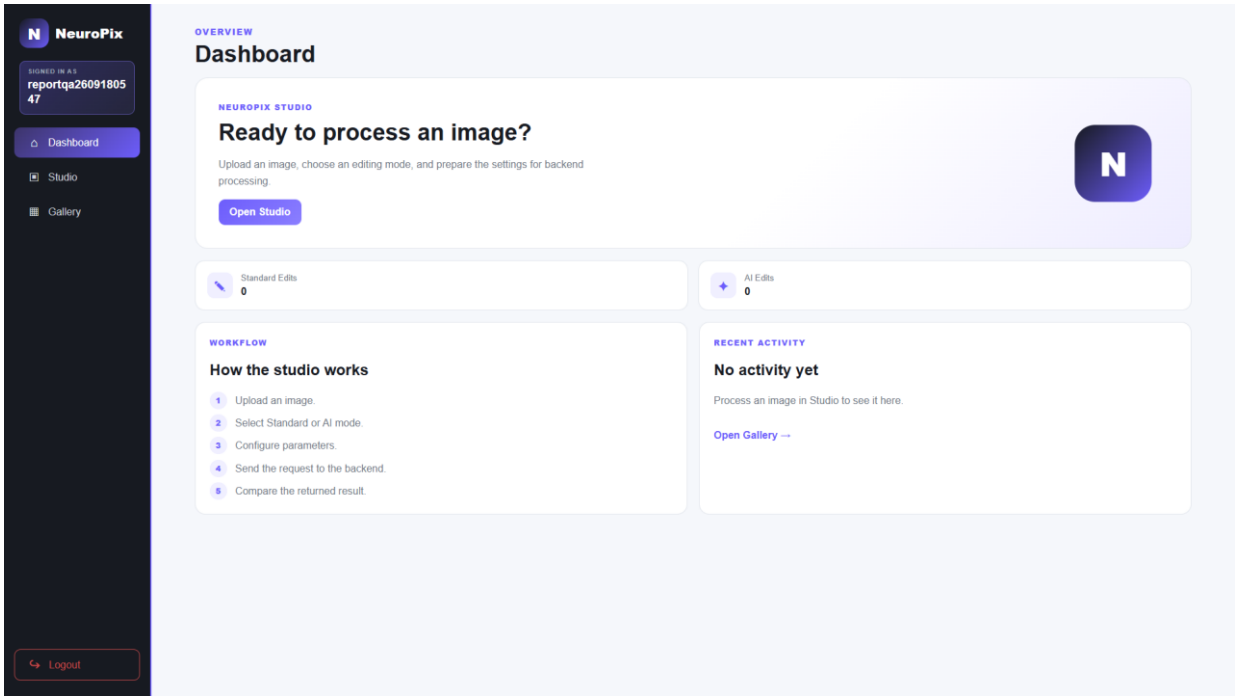
The deployed components have separate responsibilities:

- Namecheap DNS points the domain to EC2, where Nginx terminates HTTPS and forwards requests to the Flask service [1], [15]. Certbot manages the TLS certificate [1], [16]. EC2 provides the Linux virtual machine hosting these components [1], [17].
- Environment values provide database, S3, session, and provider configuration without placing credentials in the repository or frontend.
- The GitHub workflow uses a self-hosted EC2 runner to pull the application state, install requirements, restart the systemd service, and call the health endpoint [1], [18].
- The local-model service runs in Docker on the Windows computer with NVIDIA GPU access [1], [19]. The computer opens the reverse SSH tunnel used by the production application to reach that service [1], [20]. Local editing requires the tunnel and service to stay active.

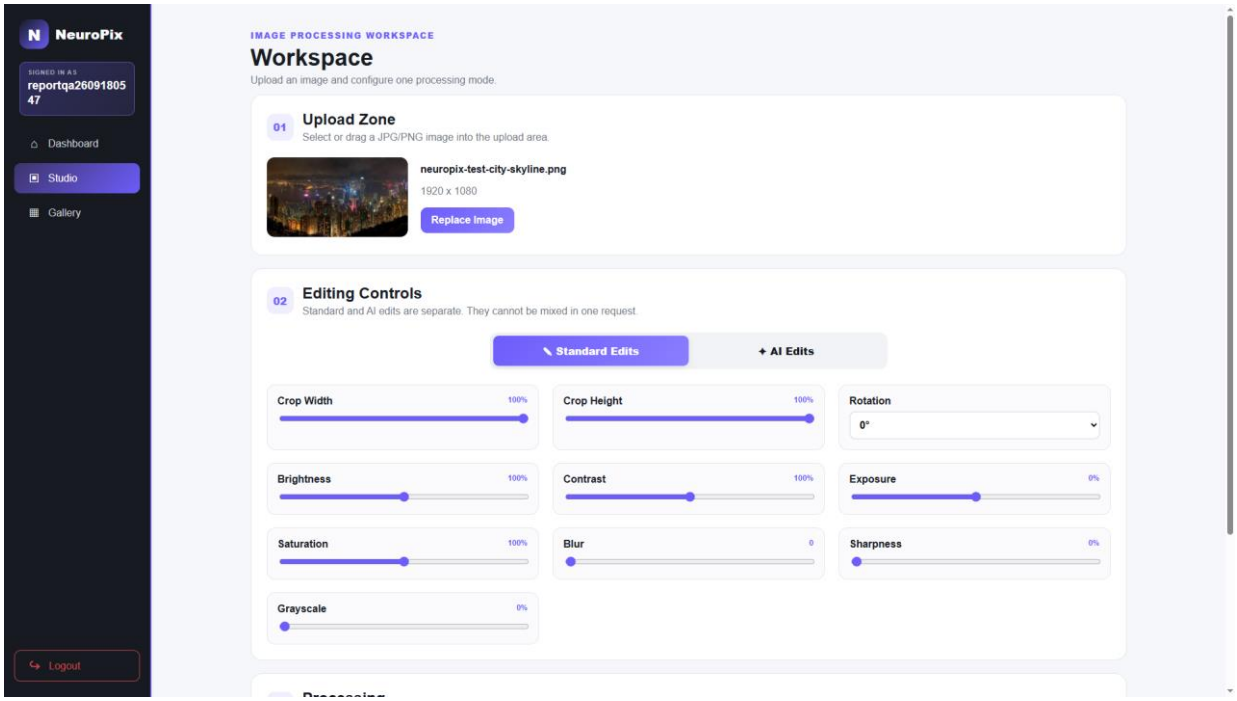
The deployment workflow is repeatable and reports a failure if the restarted application does not answer its health check. Figure 4.11 shows how the self-hosted provider connects to the production system.

5.7 Implemented Input and Output Screens

Figure 5.7-1 shows the protected dashboard and the Studio after the controlled city-skyline image was uploaded. The sidebar and section numbering provide a consistent path from overview to editing.



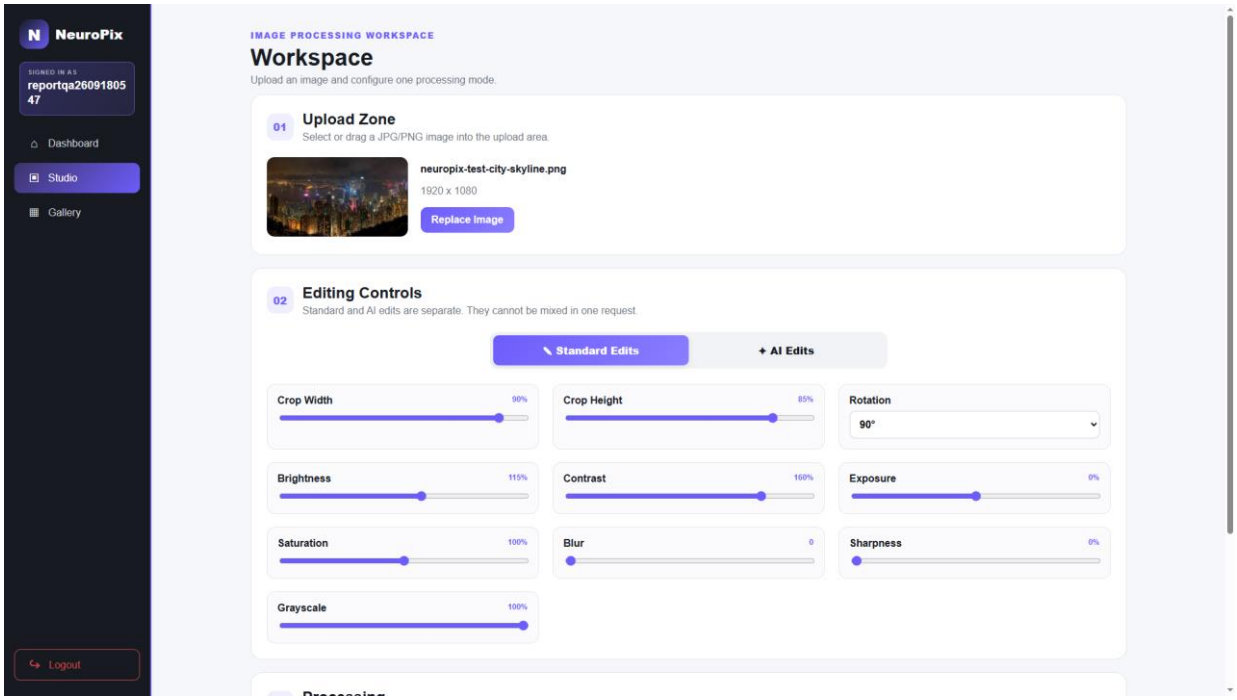
(a) Dashboard



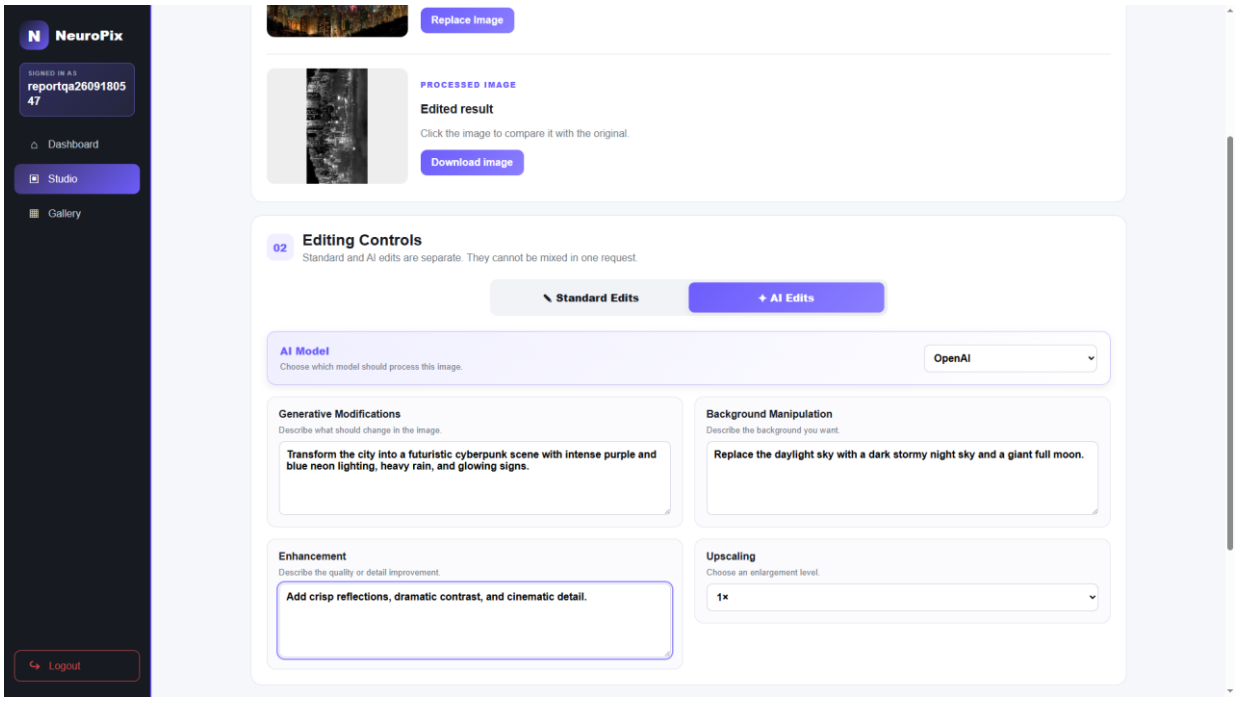
(b) Studio upload workspace

Figure 5.7-1: Dashboard and Studio upload.

The Standard controls expose numeric settings, while the AI panel replaces them with provider and purpose-specific text fields. The interface shows the controls for the selected mode, keeping Standard settings separate from AI instructions.



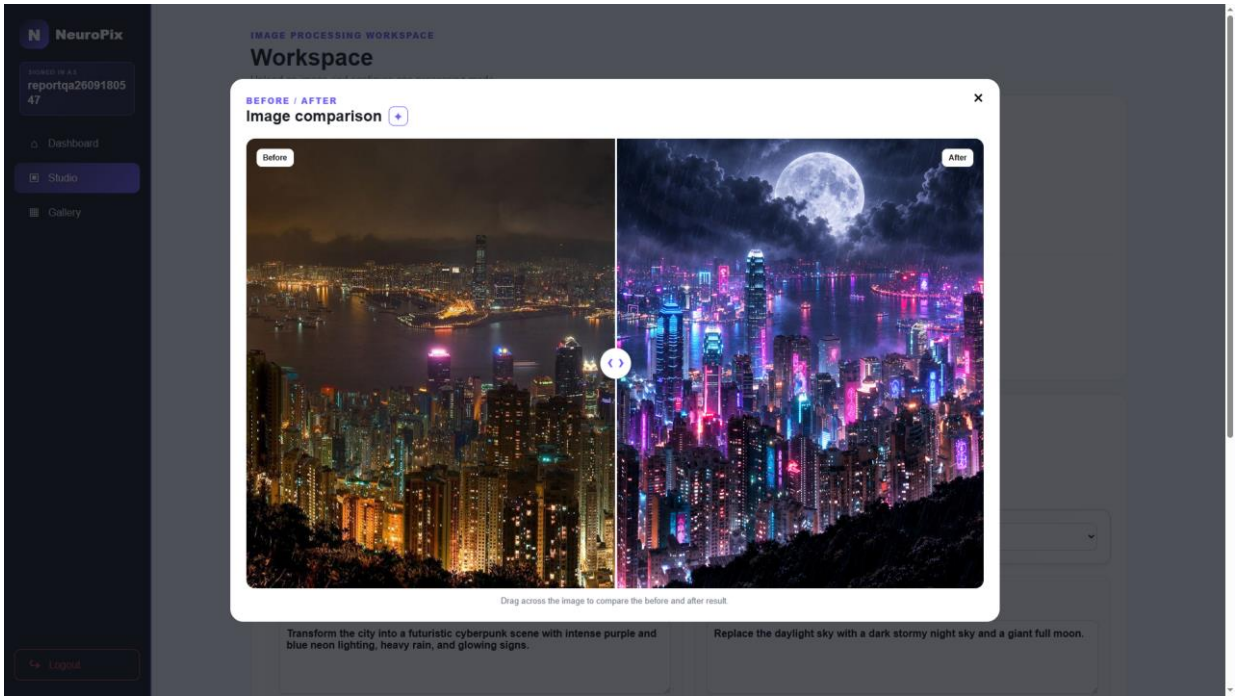
(a) Standard editing controls



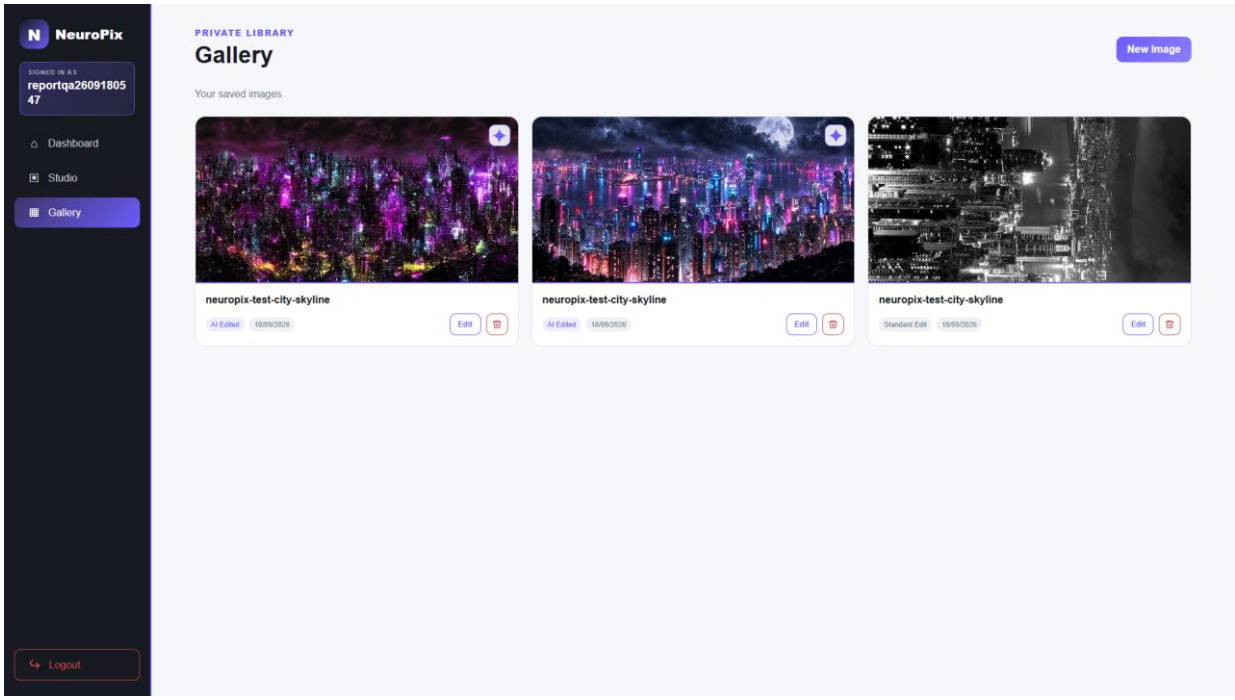
(b) OpenAI editing controls

Figure 5.7-2: Standard and OpenAI controls.

The processed output can be compared directly with its source, and saved results appear in the private gallery with their edit type and management actions.



(a) Before-and-after comparison



(b) Personal gallery

Figure 5.7-3: AI comparison and gallery.

Chapter 6 Testing and Evaluation

6.1 Testing Strategy

Testing focused on the main user workflows. A compact automated suite checked selected API and security behavior, while six live use cases exercised the deployed site in Chrome. Each use case defined preconditions and expected results, then recorded the observed actions and outcome. Sequence diagrams explain the interactions, and screenshots provide evidence of the visible behavior.

The automated suite was rerun from the project repository on October 10, 2026 after switching the signed-in pages to Jinja rendering. It covered authentication, upload validation, processing responses, password hashing, selected local-provider behavior, signed-in page rendering and redirects, gallery downloads, and mocked storage coordination. These tests do not establish cross-account gallery isolation or verify the actual SQL queries.

A local browser check on October 10, 2026 used real MySQL and S3 services. A PNG upload was processed, saved, and shown in the Dashboard and Gallery. Both versions downloaded, and the edited version reopened in Studio. Deleting the test image removed its database record and both S3 files without changing other records.

Table 6.1: Automated test result.

Test scope	Command and result
API, security, and page checks	python -m pytest -q tests/test_api.py tests/test_security.py tests/test_gallery_download.py tests/test_gallery_page.py tests/test_signed_in_pages.py. All 35 tests passed

6.2 Live Test Environment and Controlled Data

Table 6.2: Live use-case test environment.

Item	Verified configuration
Site	https://neuropix.me production deployment over HTTPS
Browser	Google Chrome controlled through Playwright CLI
Viewport	1920 × 1080 at normal browser zoom
Execution date	September 18, 2026
Account	New dedicated test account created through UC-T01
Image	neuropix-test-city-skyline.png, PNG, 1920 × 1080
Standard settings	Crop 90% × 85%, rotation 90°, brightness 115%, contrast 160%, and grayscale 100%
AI instruction	Extreme cyberpunk night transformation with neon lighting, rain, a stormy sky, a large moon, reflections, and cinematic detail

The city skyline was chosen because geometry, grayscale, lighting, background, and generative changes are easy to recognize. The same staged image was reused after UC-T03 so UC-T04 and UC-T05 did not repeat login and upload.

6.3 Evidence and Completion Rules

A case passed when we observed its expected visible results. Screenshots recorded the interface, and sequence diagrams explained the implementation, but this evidence could not confirm every internal storage or database operation. AI cases also required a returned image, and gallery deletion required the selected card to remain absent after refresh.

Testing was complete when all six cases had a recorded outcome supported by live evidence. Each result covers only the stated preconditions and actions. Other features and failure paths need separate tests.

6.4 UC-T01: Register a New User

Precondition: the test username does not exist and the browser has no authenticated session. The user opens registration, enters a unique username and matching valid password, and submits the form. The expected result is successful account creation followed by the login page.

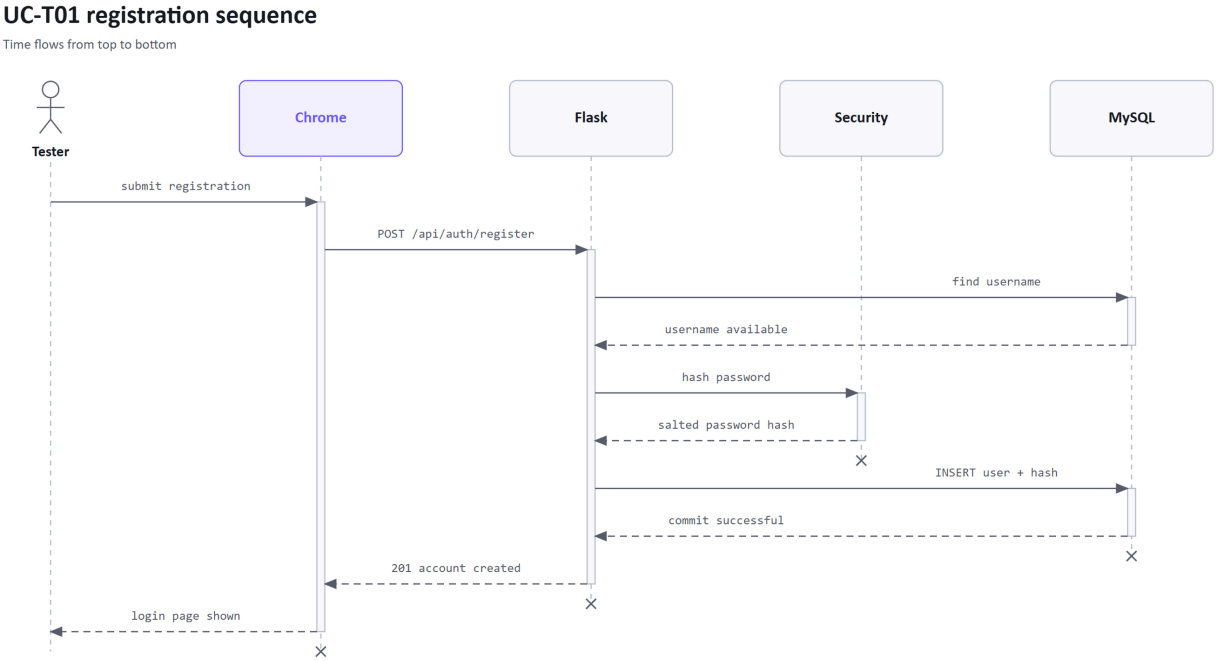
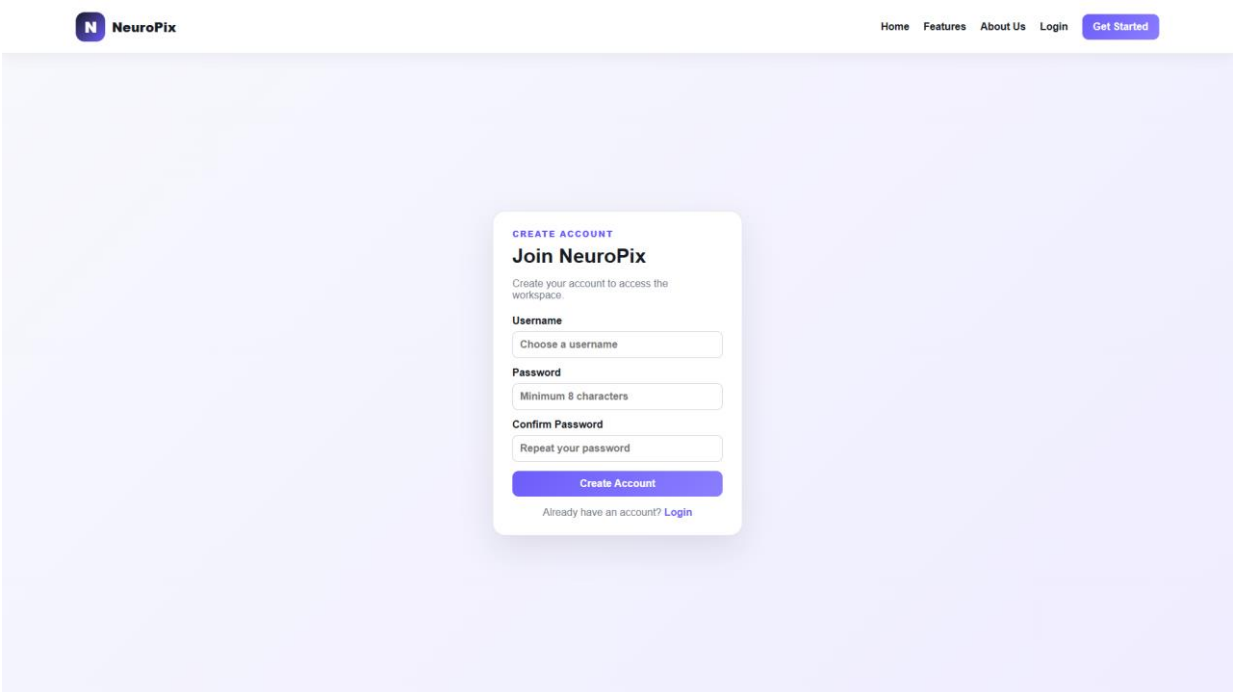


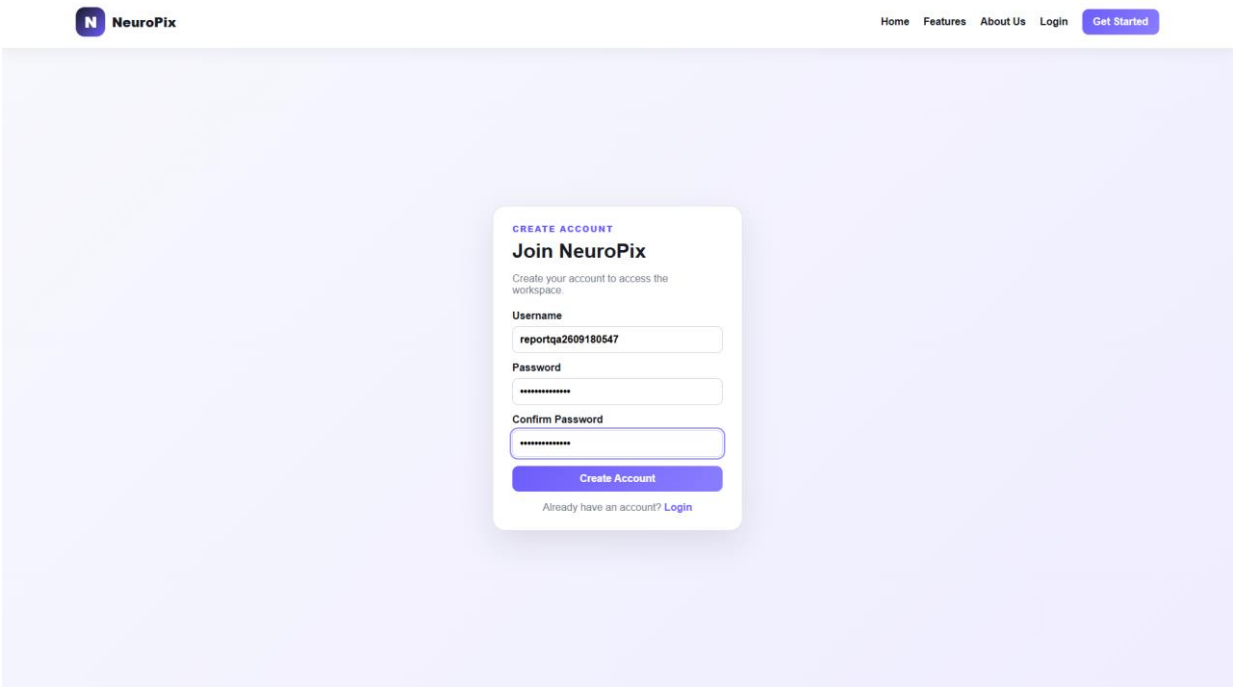
Figure 6.4-1: UC-T01 registration sequence.

Table 6.4: UC-T01 execution result.

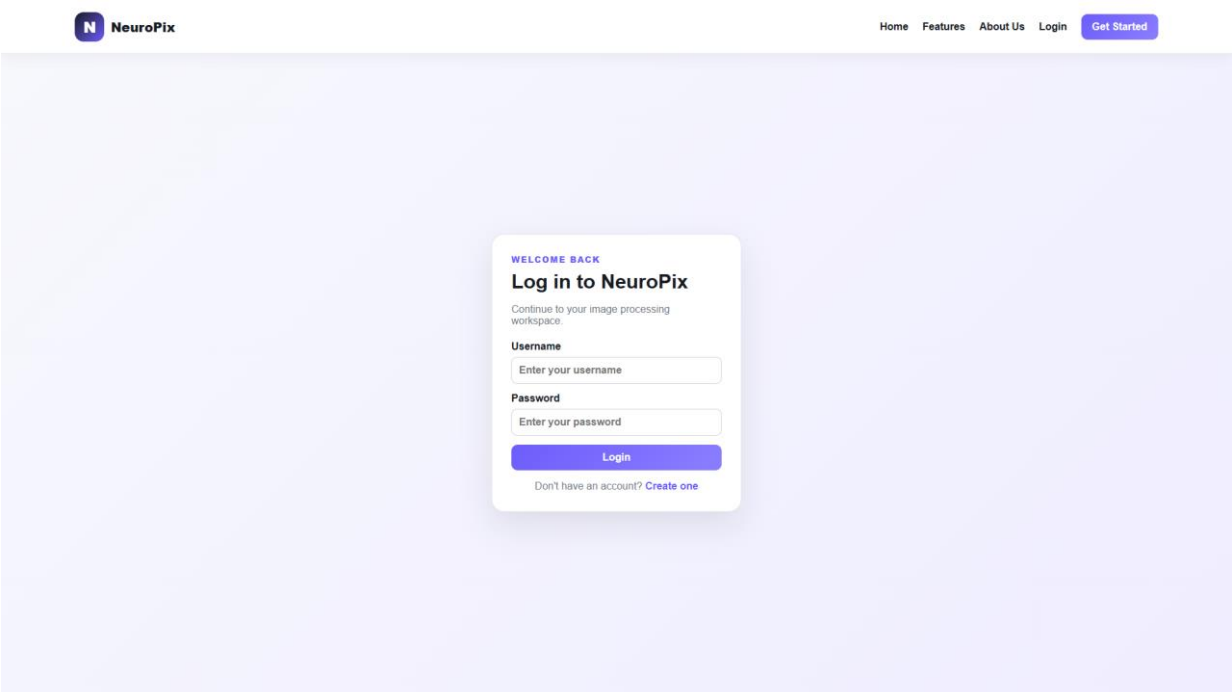
Step	Observed result
Open registration	Registration form displayed with username, password, and confirmation fields
Enter valid data	Username remained visible and both password fields remained masked
Submit	The site reported that the account was created and redirected to login
Final status	**Pass**



UC-T01 (a) Registration form



UC-T01 (b) Registration details entered



UC-T01 (c) Account created and login page opened

Figure 6.4-2: UC-T01 registration evidence.

6.5 UC-T02: Login and Access a Protected Page

Precondition: the UC-T01 account exists and the user is on the login page. The user enters valid credentials and submits. The expected result is an authenticated session and access to the dashboard.

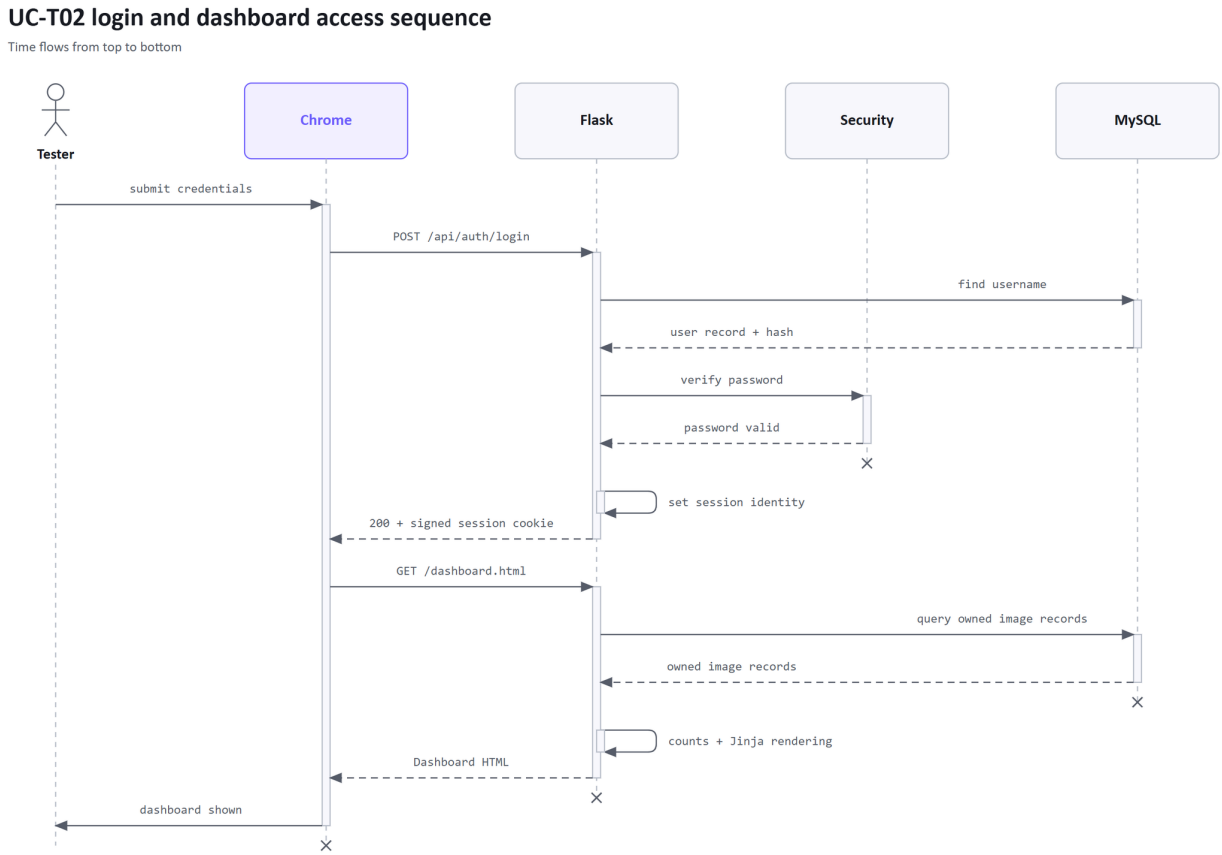
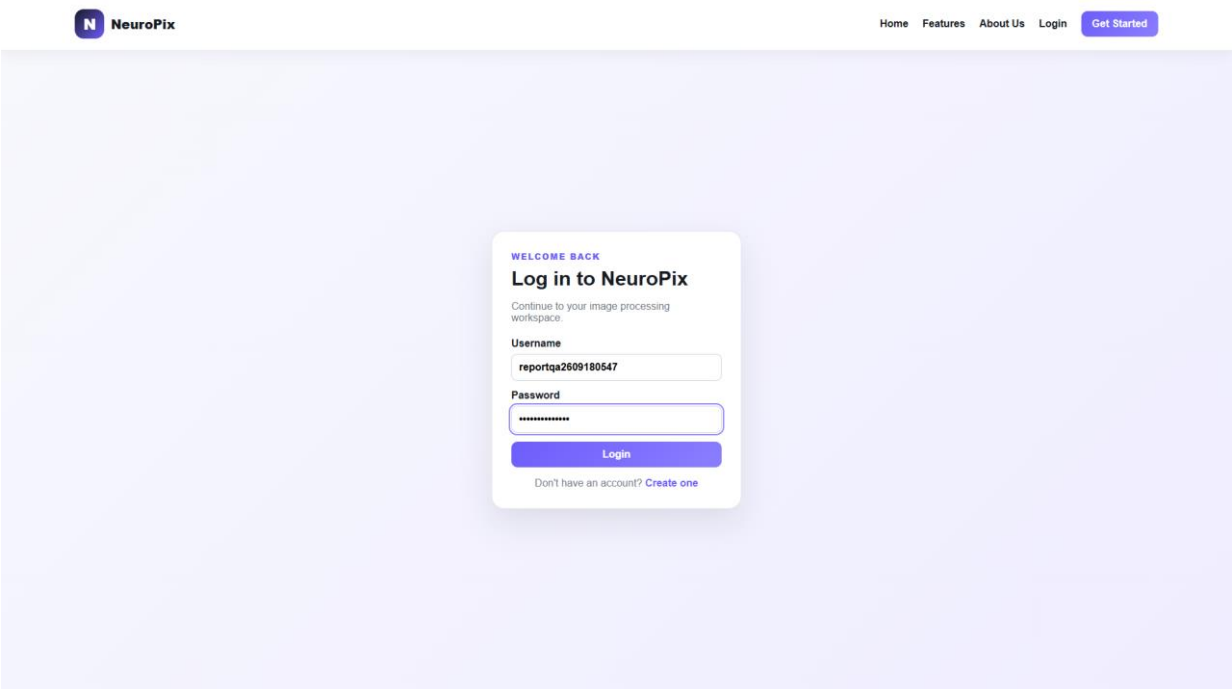


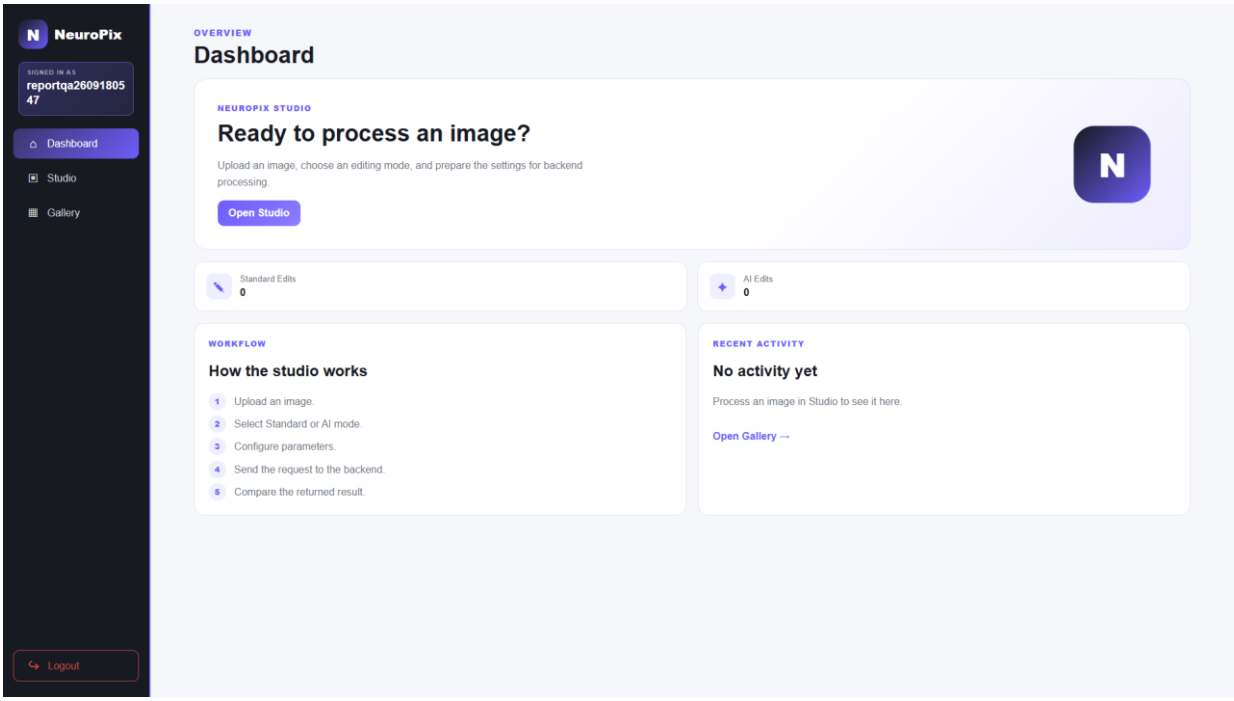
Figure 6.5-1: UC-T02 login and protected-access sequence.

Table 6.5: UC-T02 execution result.

Step	Observed result
Enter credentials	Username displayed and password remained masked
Submit login	Dashboard opened and showed the signed-in username, private navigation, and empty initial counts
Final status	**Pass**



UC-T02 (a) Login credentials entered



UC-T02 (b) Authenticated dashboard

Figure 6.5-2: UC-T02 login evidence.

6.6 UC-T03: Upload and Apply a Standard Edit

Precondition: UC-T02 has established the session. The user opens the Studio and uploads the controlled image. Cropping, rotation, and maximum grayscale make the changes easy to see. The expected result is a processed image available for comparison.

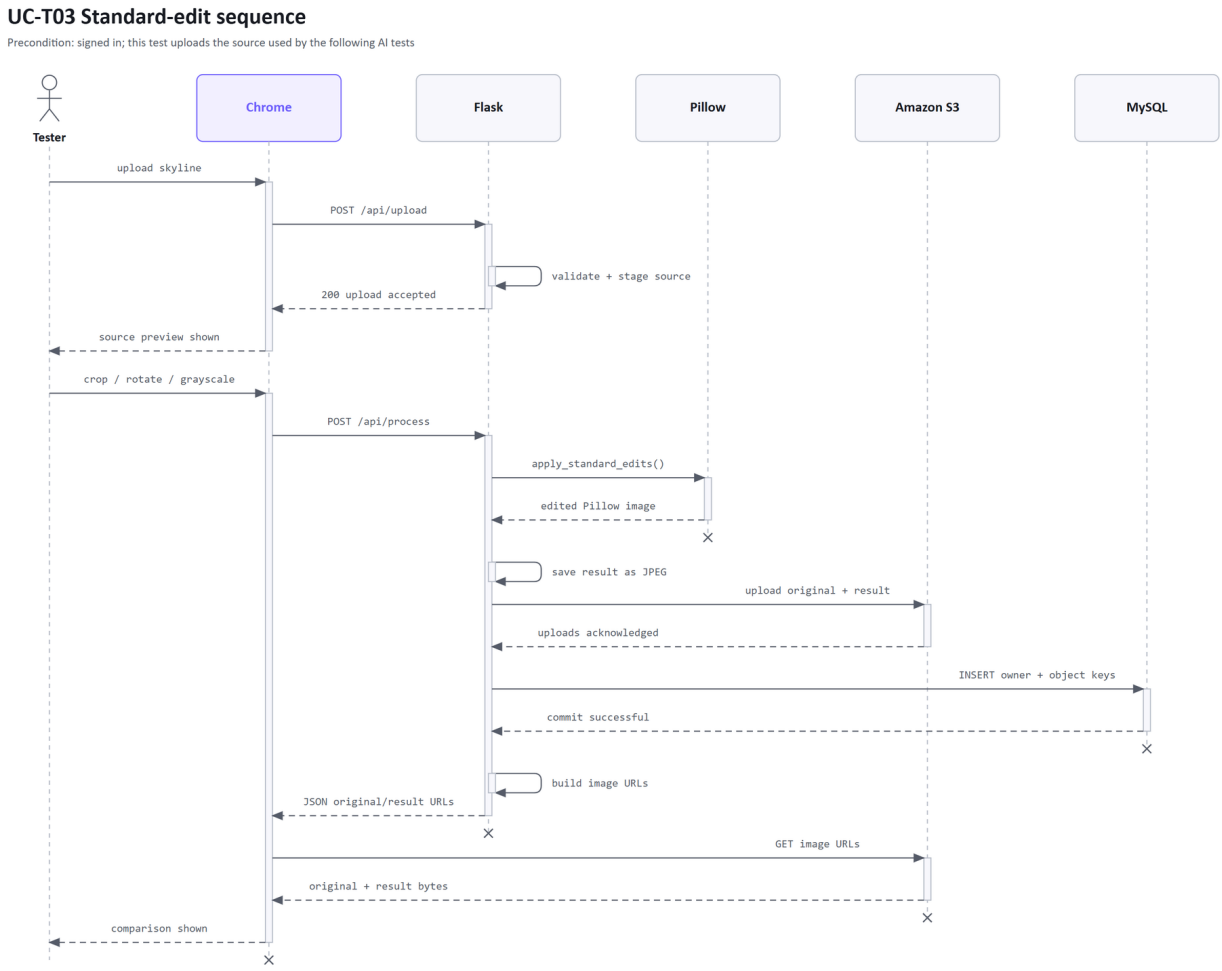
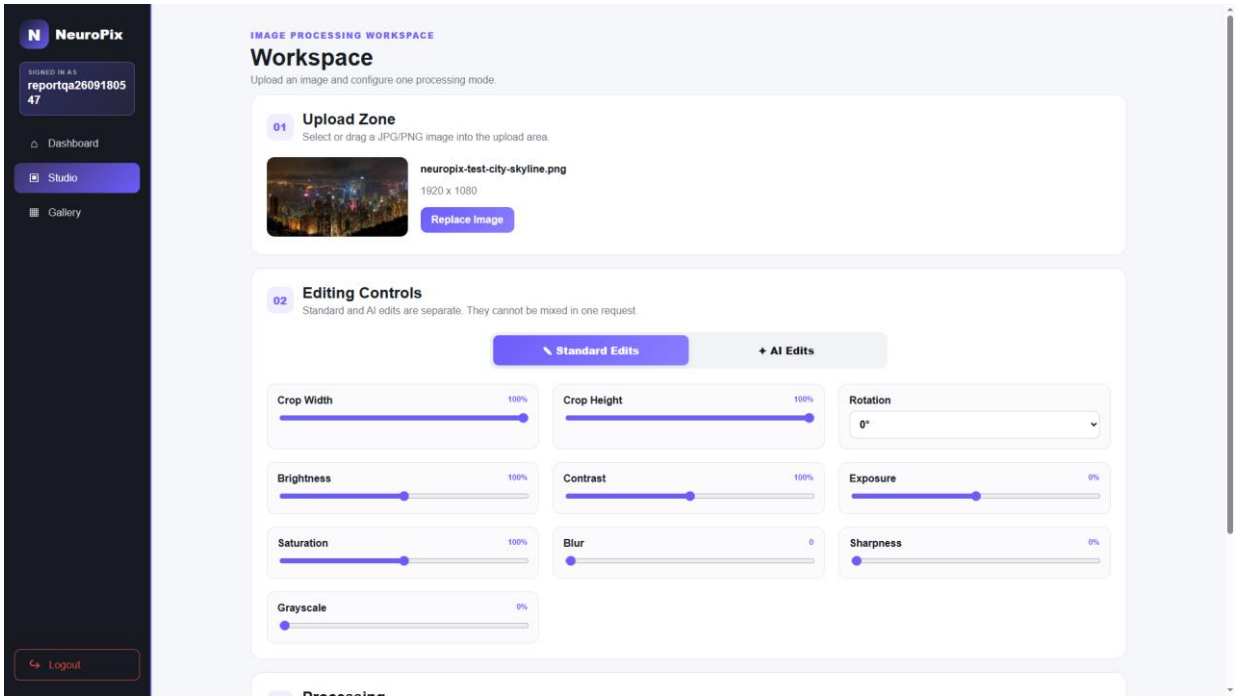


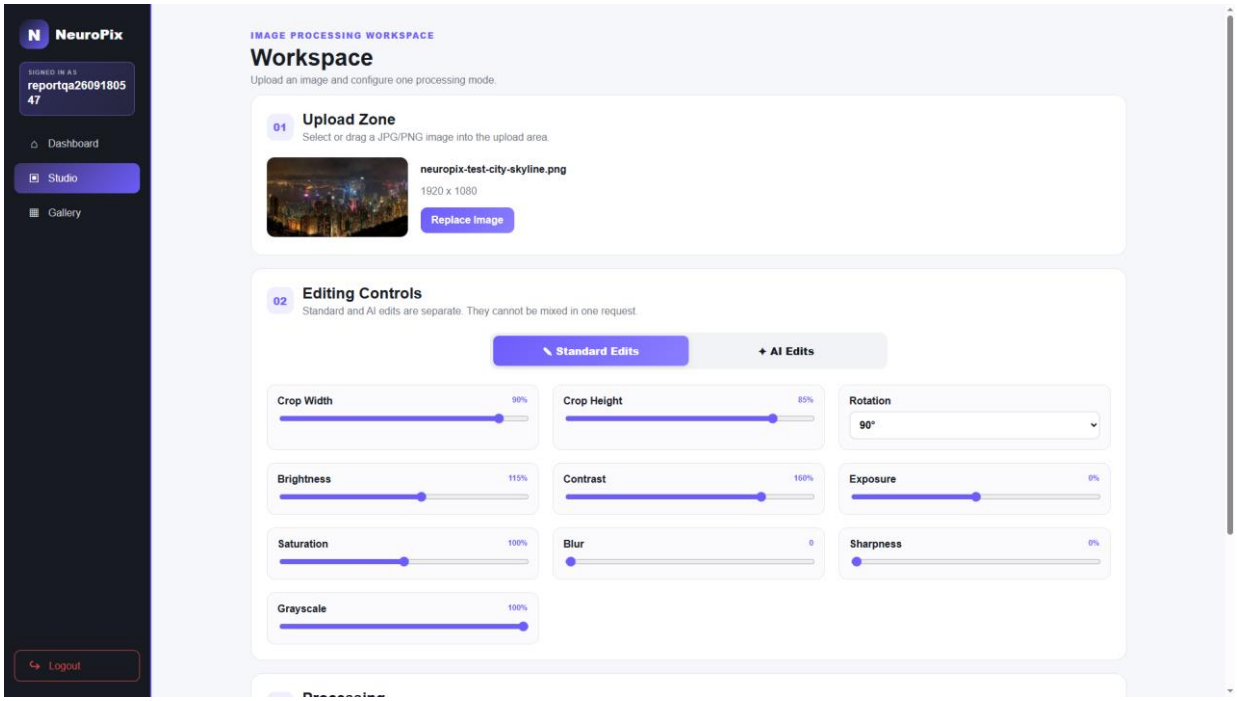
Figure 6.6-1: UC-T03 upload and Standard-edit sequence.

Table 6.6: UC-T03 execution result.

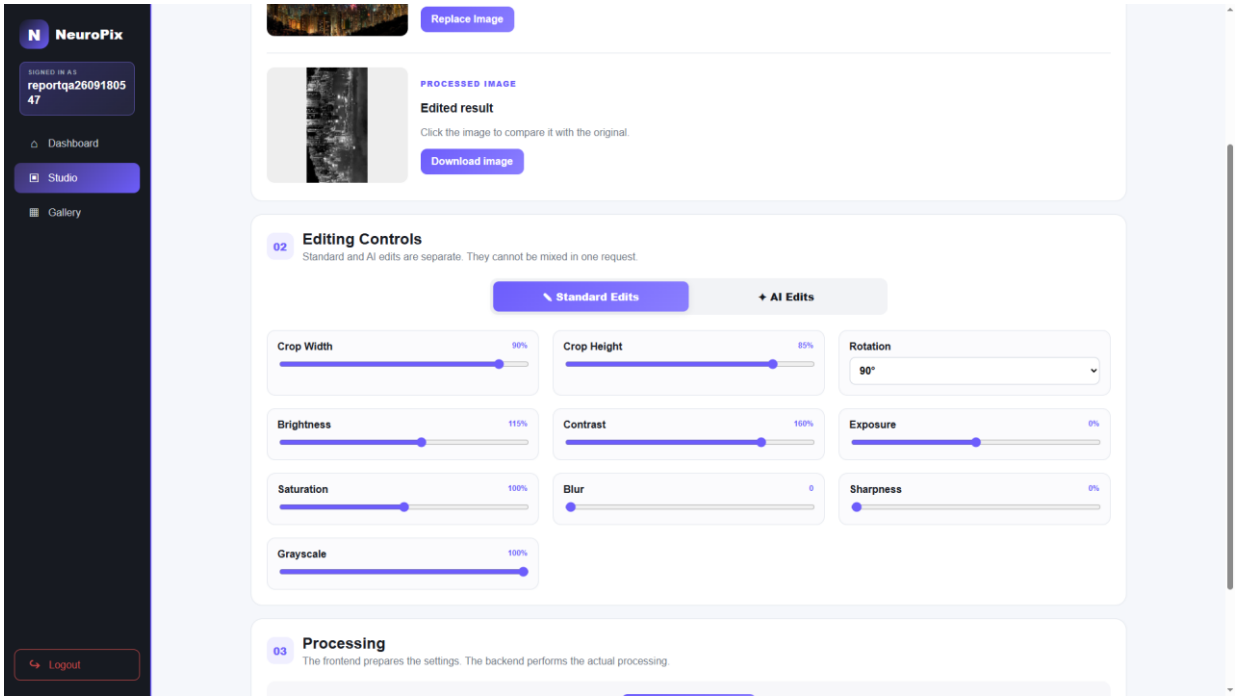
Step	Observed result
Upload	The source preview displayed the correct filename and 1920 × 1080 dimensions
Configure	All selected values appeared beside their controls, including 100% grayscale
Process	A processed preview and success message appeared
Compare	The modal showed the original beside the rotated, cropped, high-contrast grayscale result
Final status	**Pass**



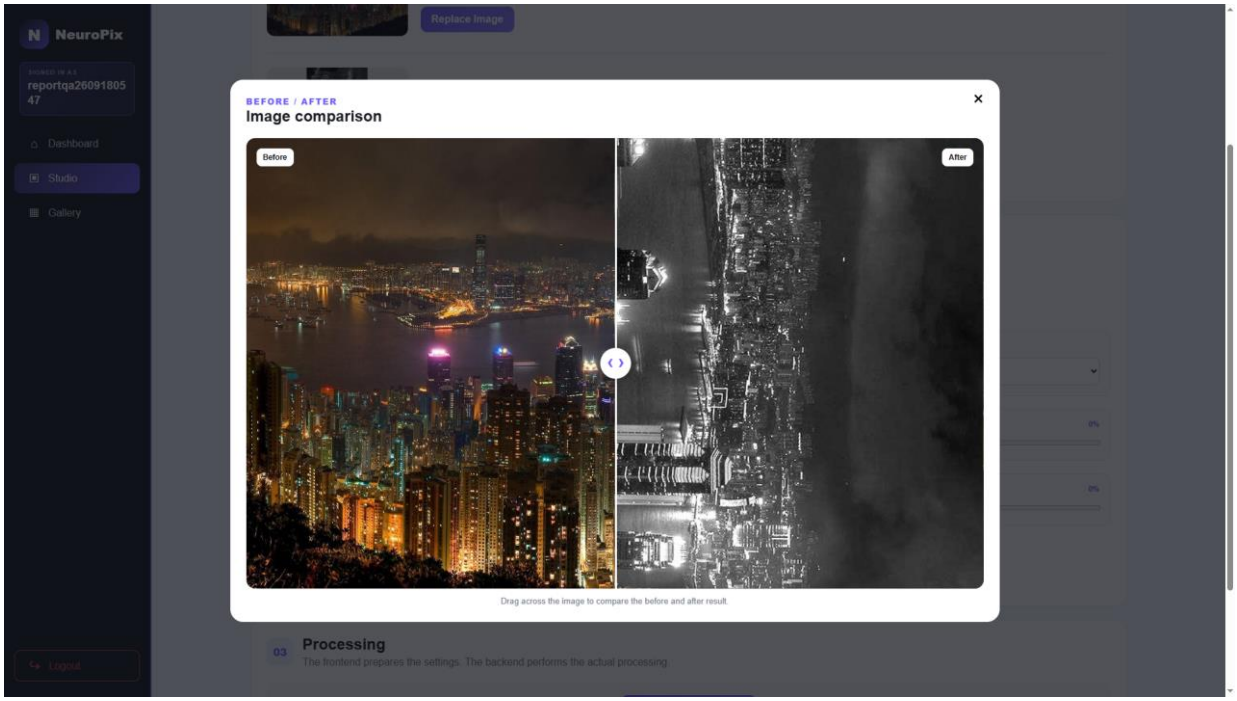
UC-T03 (a) City skyline uploaded



UC-T03 (b) Standard settings selected



UC-T03 (c) Processed result



UC-T03 (d) Before-and-after comparison

Figure 6.6-2: UC-T03 Standard-edit evidence.

6.7 UC-T04: Edit with OpenAI

Precondition: the user is authenticated and the controlled image is staged. The user selects OpenAI and enters content, background, and enhancement instructions. The expected result is a new image available for comparison, with no repeated login or upload.

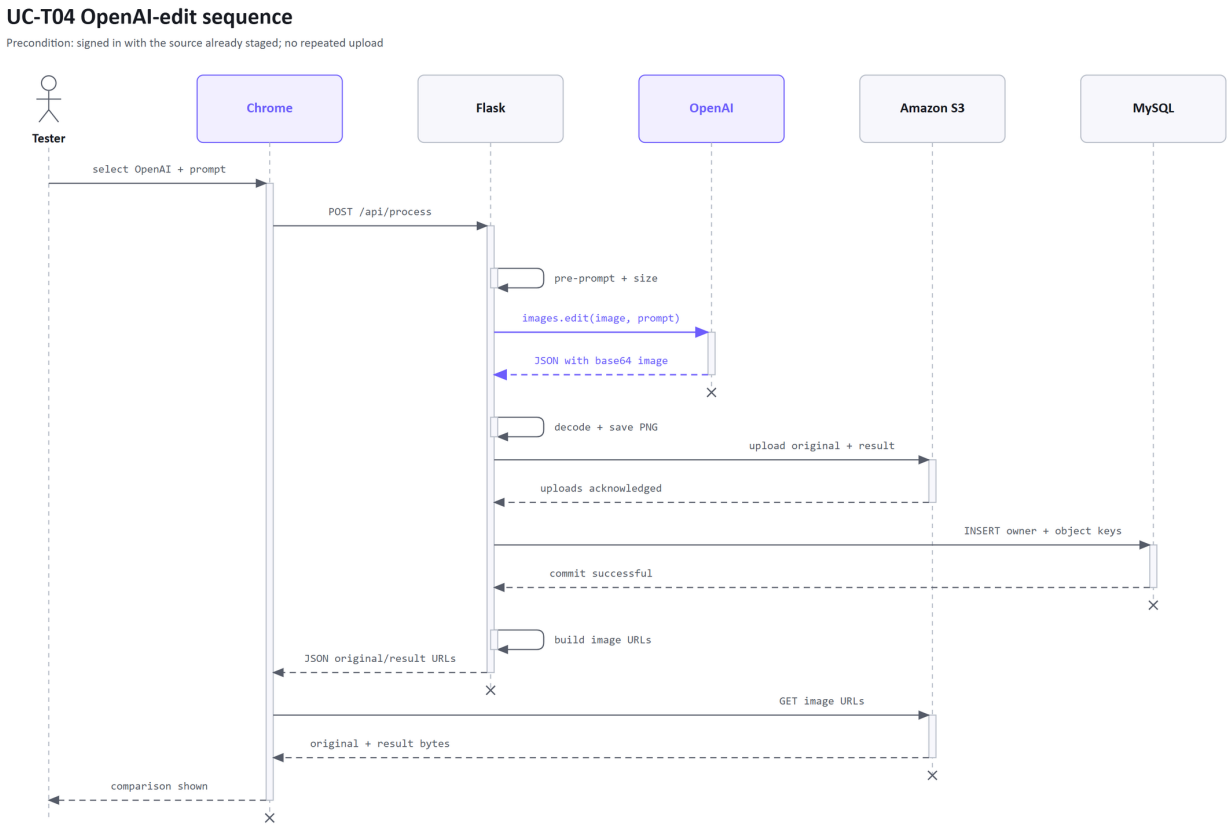
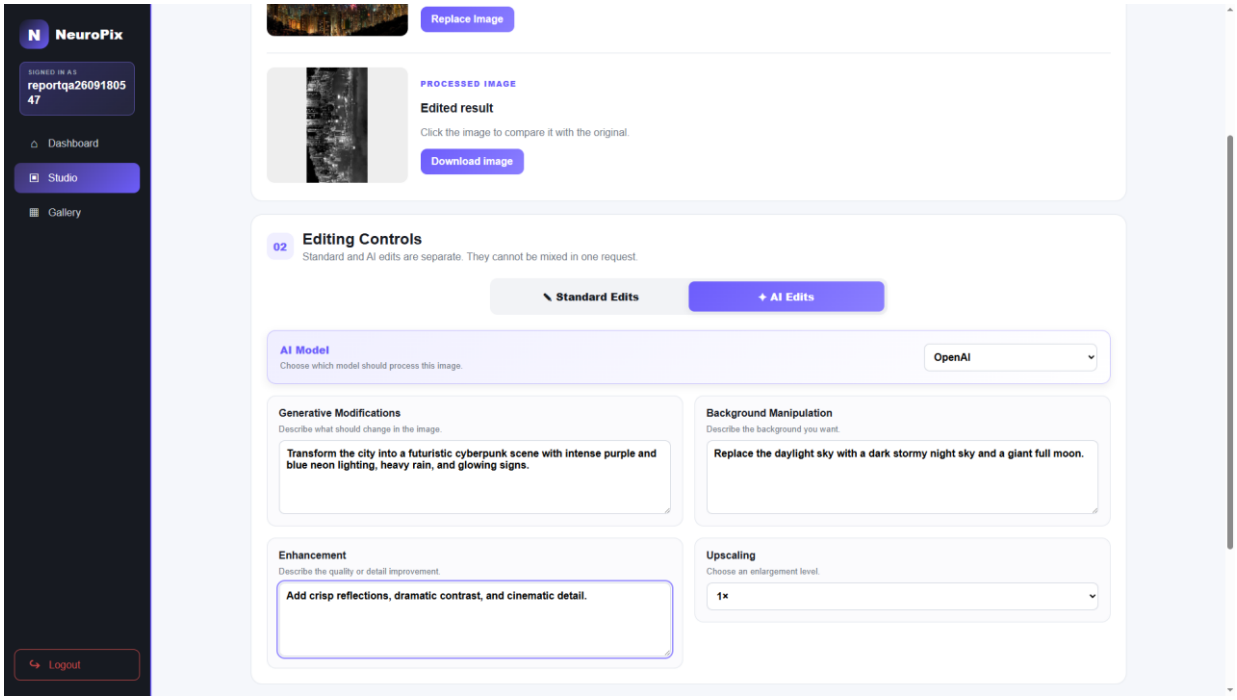


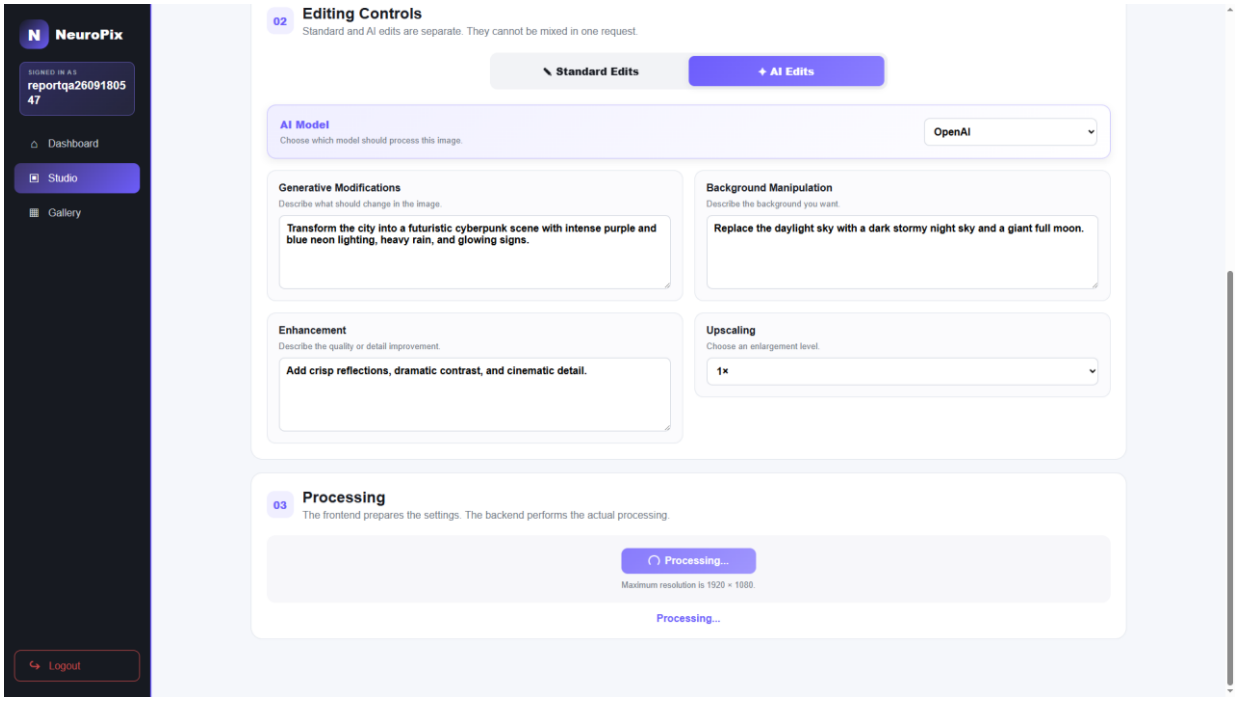
Figure 6.7-1: UC-T04 OpenAI-edit sequence.

Table 6.7: UC-T04 execution result.

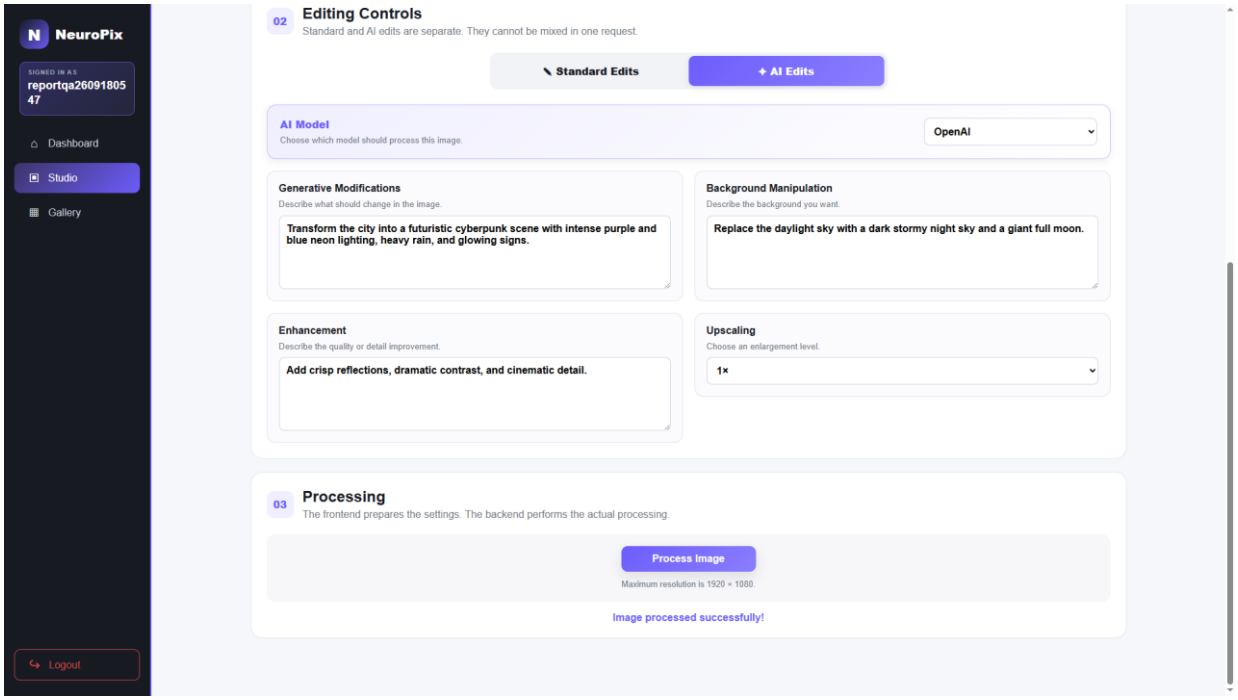
Step	Observed result
Configure	OpenAI was the selected provider and all three purpose-specific fields retained the instructions
Submit	The Process button changed to a disabled processing state
Return result	The success state appeared and the processed preview was replaced
Compare	The result showed an obvious neon night city with rain, a large moon, and altered lighting while retaining the broad skyline composition
Final status	**Pass**



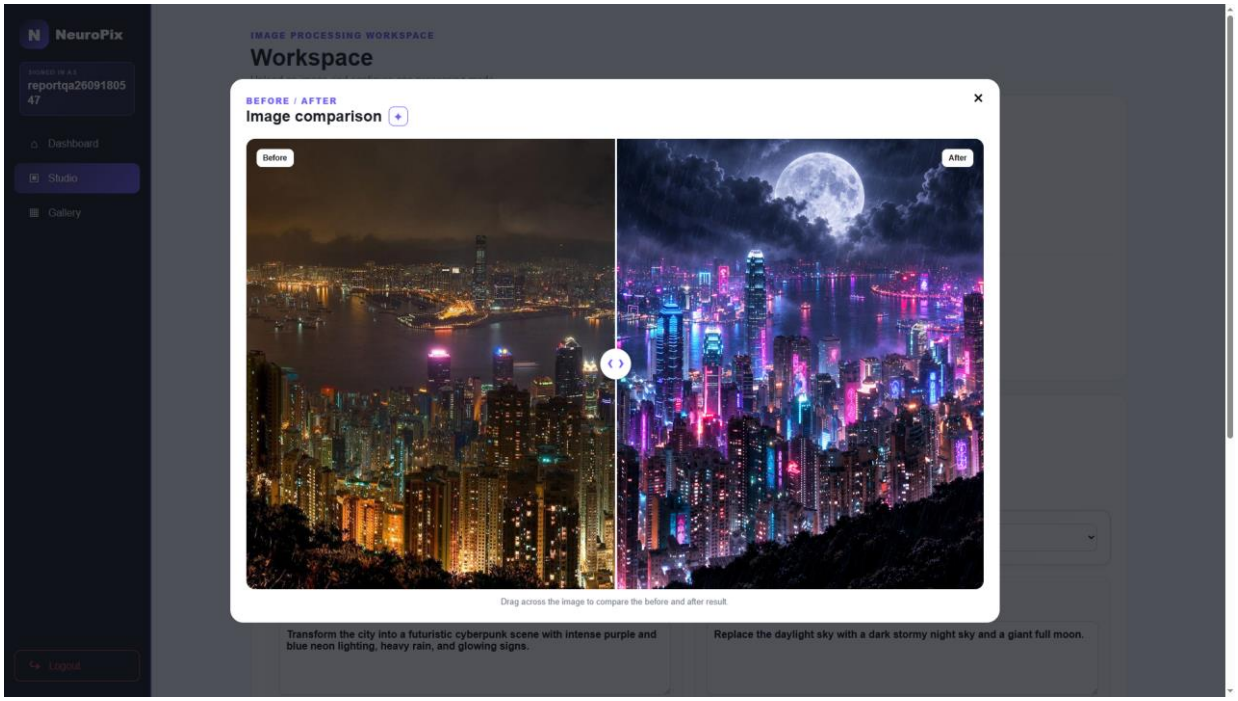
UC-T04 (a) OpenAI instructions entered



UC-T04 (b) OpenAI processing



UC-T04 (c) OpenAI result returned



UC-T04 (d) Before-and-after comparison

Figure 6.7-2: UC-T04 OpenAI-edit evidence.

6.8 UC-T05: Edit with Local Stable Diffusion

Precondition: the user is authenticated, the controlled source is staged, and the local computer, Docker service, and reverse SSH tunnel are active. The availability check must report the local provider as ready. The user selects Local Stable Diffusion and submits the same instructions used for OpenAI. The expected result is a new image available for comparison.

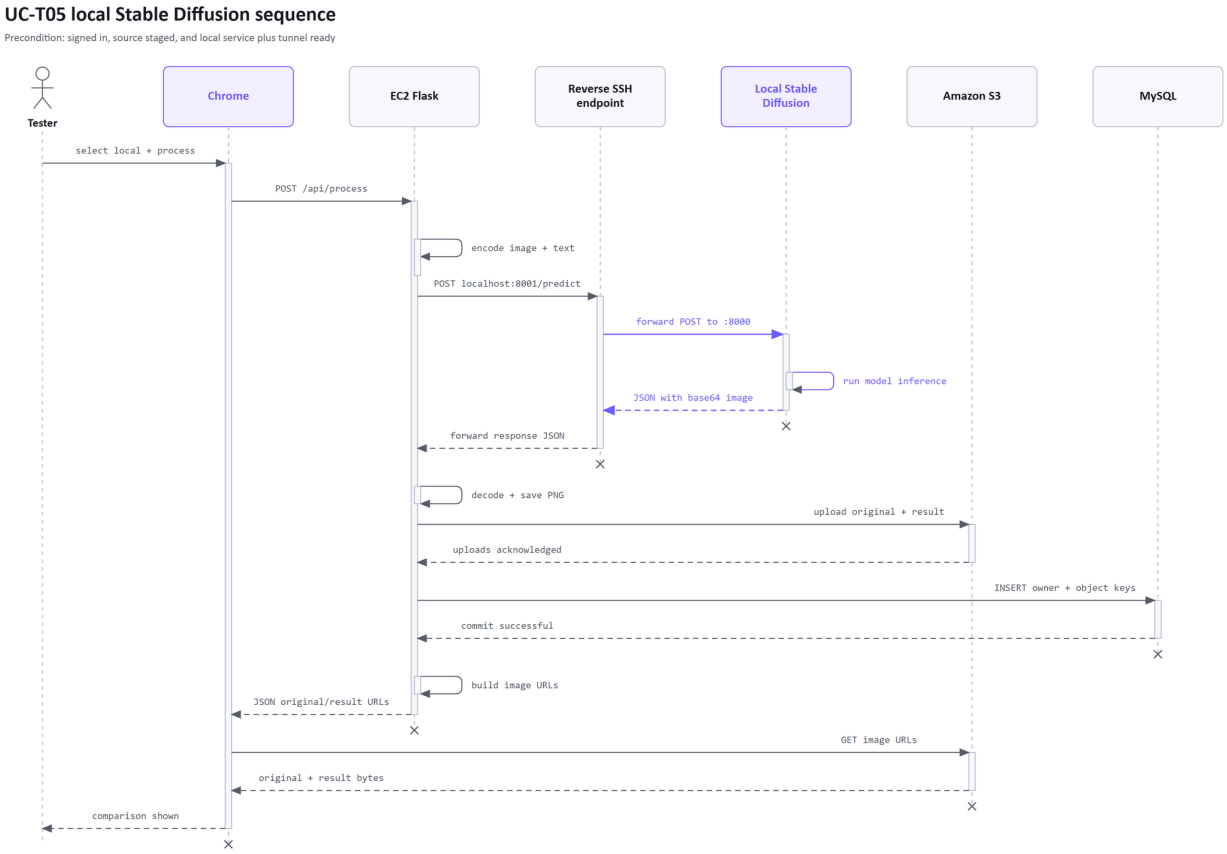
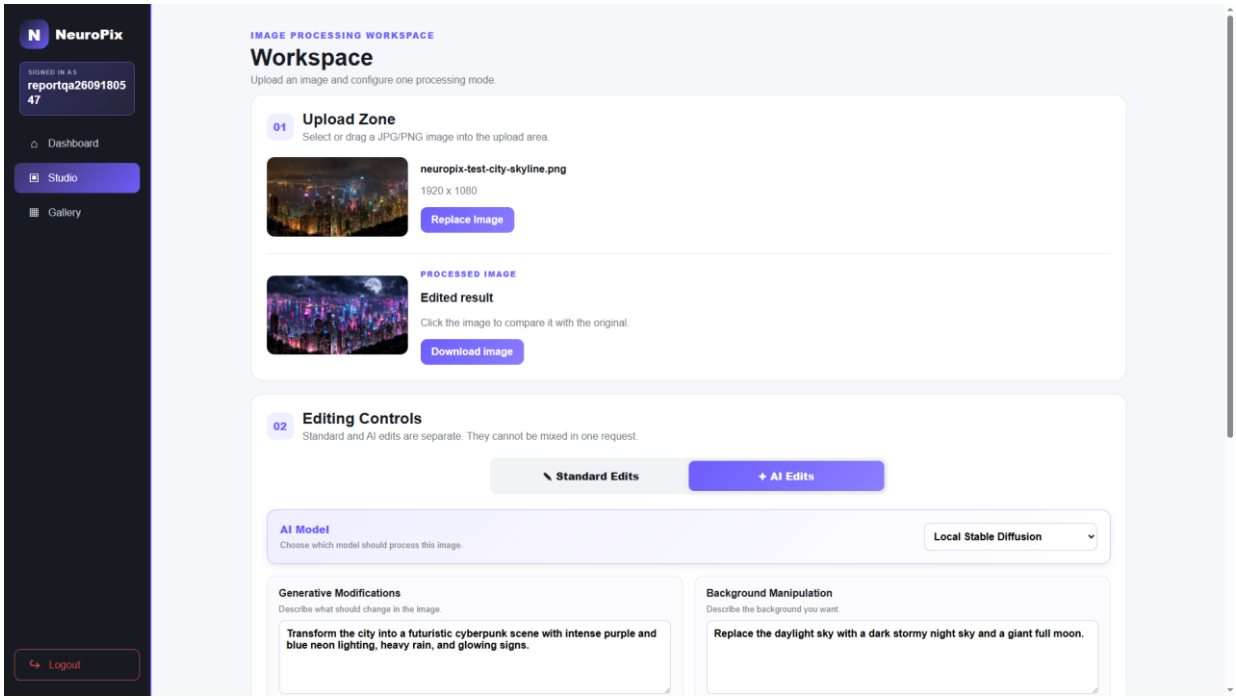


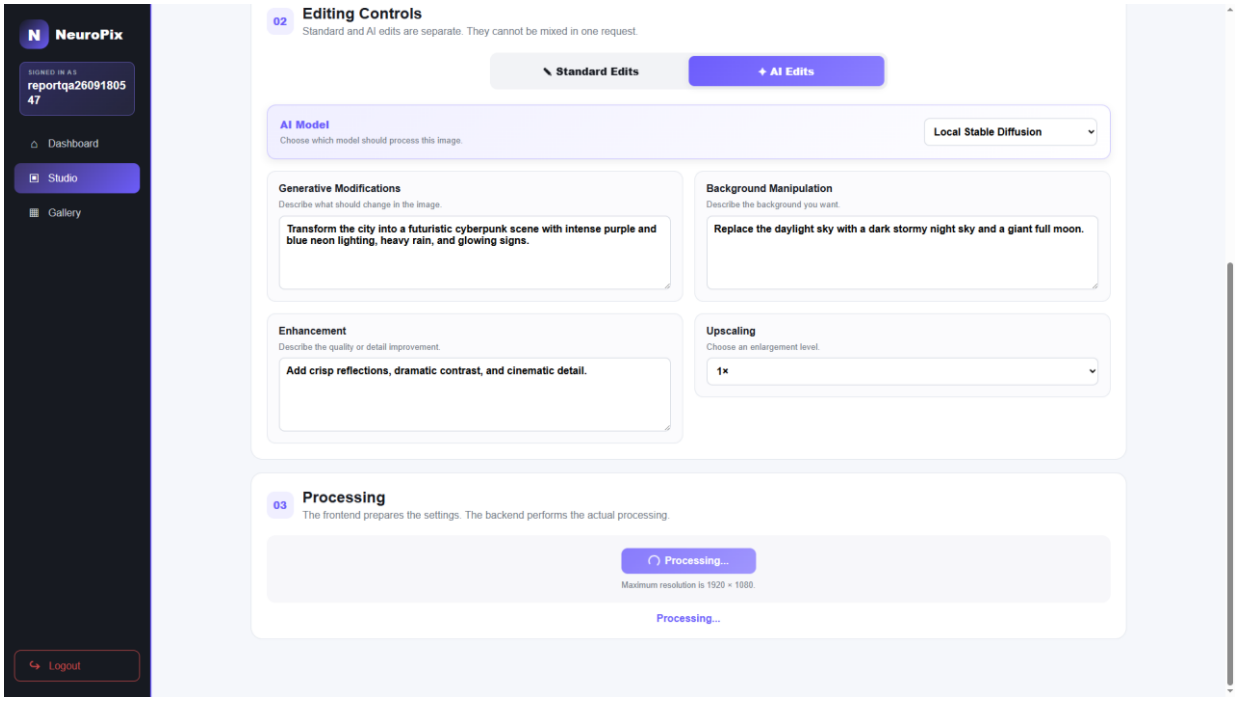
Figure 6.8-1: UC-T05 local Stable Diffusion sequence.

Table 6.8: UC-T05 execution result.

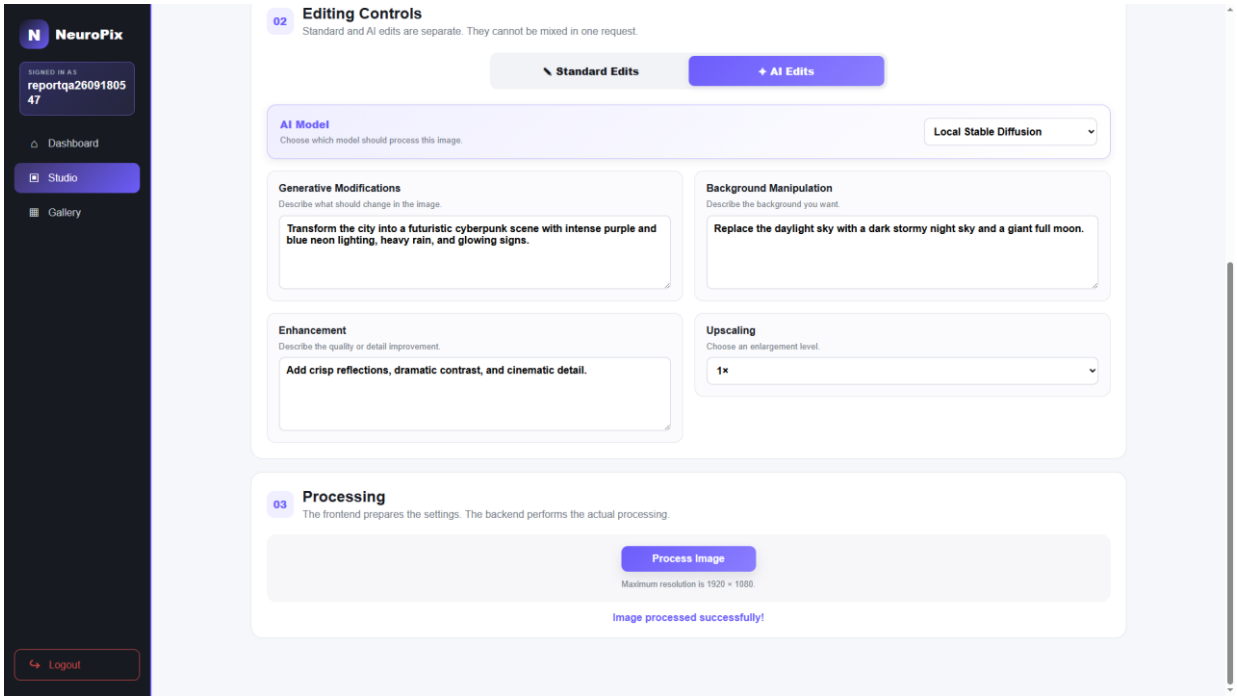
Step	Observed result
Select provider	Local Stable Diffusion appeared as the active provider
Submit	The request entered the disabled processing state while the local service generated the image
Return result	The site reported success and displayed a new processed preview
Compare	The local result produced a strong purple futuristic city transformation that was visibly different from both the source and hosted result
Final status	**Pass**



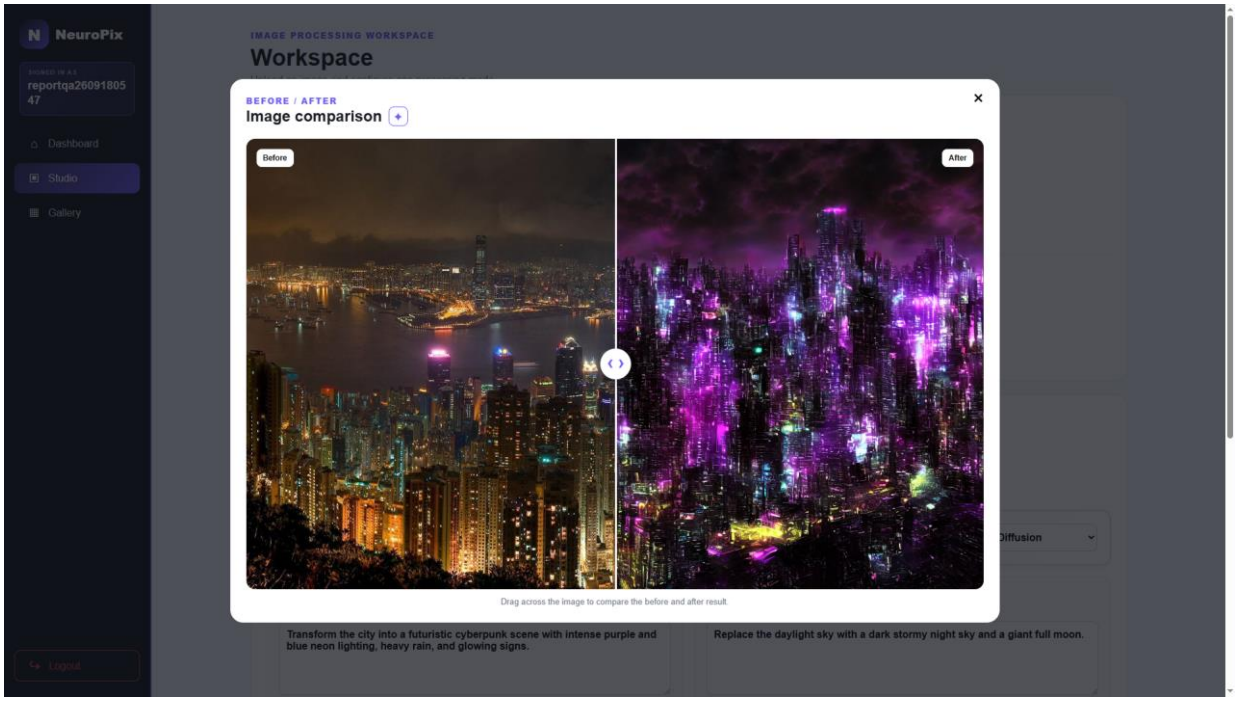
UC-T05 (a) Local provider selected



UC-T05 (b) Local processing



UC-T05 (c) Local result returned



UC-T05 (d) Before-and-after comparison

Figure 6.8-2: UC-T05 local-model evidence.

6.9 UC-T06: Verify and Delete a Gallery Image

Precondition: UC-T03, UC-T04, and UC-T05 have created saved results for the current user. The user verifies the latest gallery image, confirms deletion, and refreshes the page. The selected image should remain absent while the other saved results remain visible.

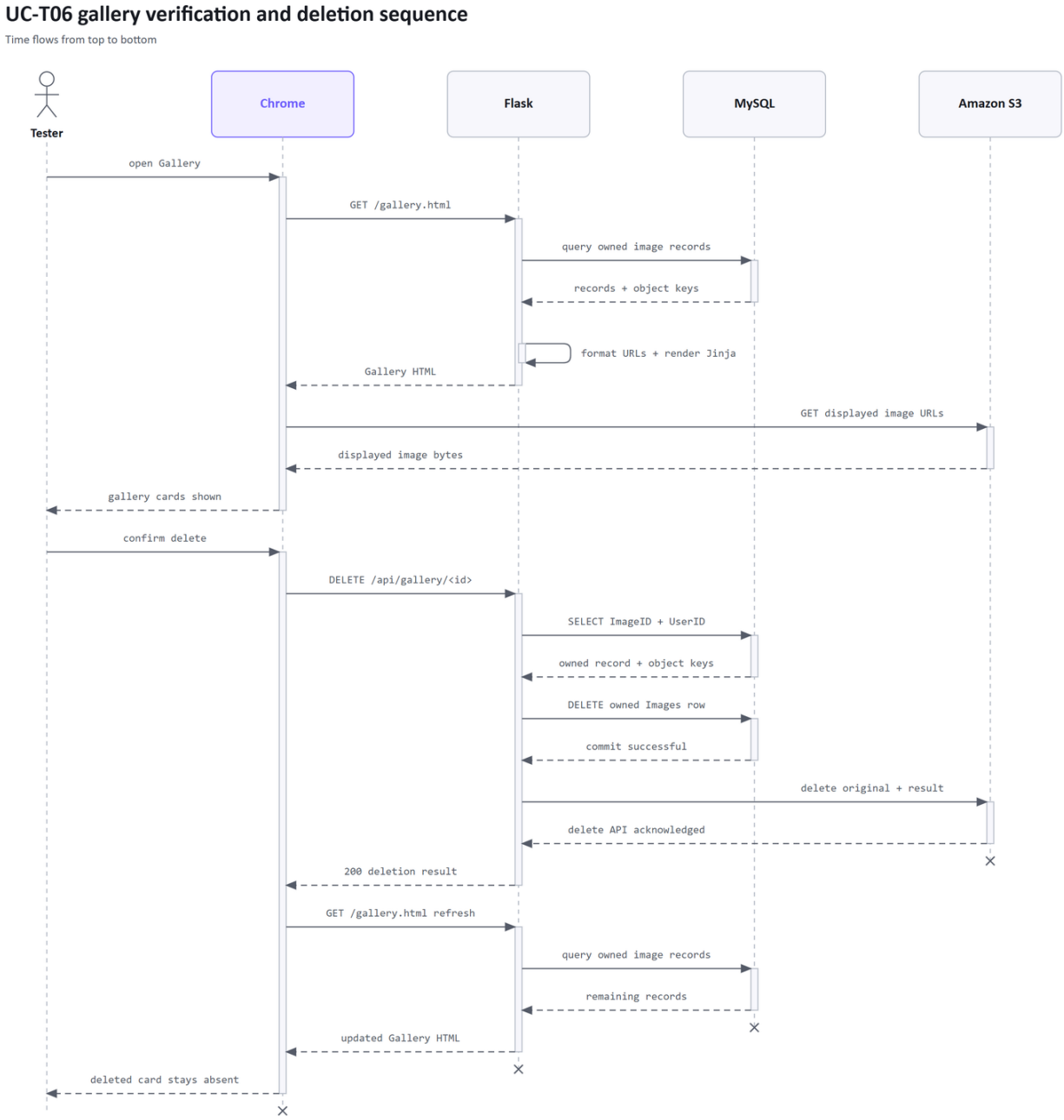
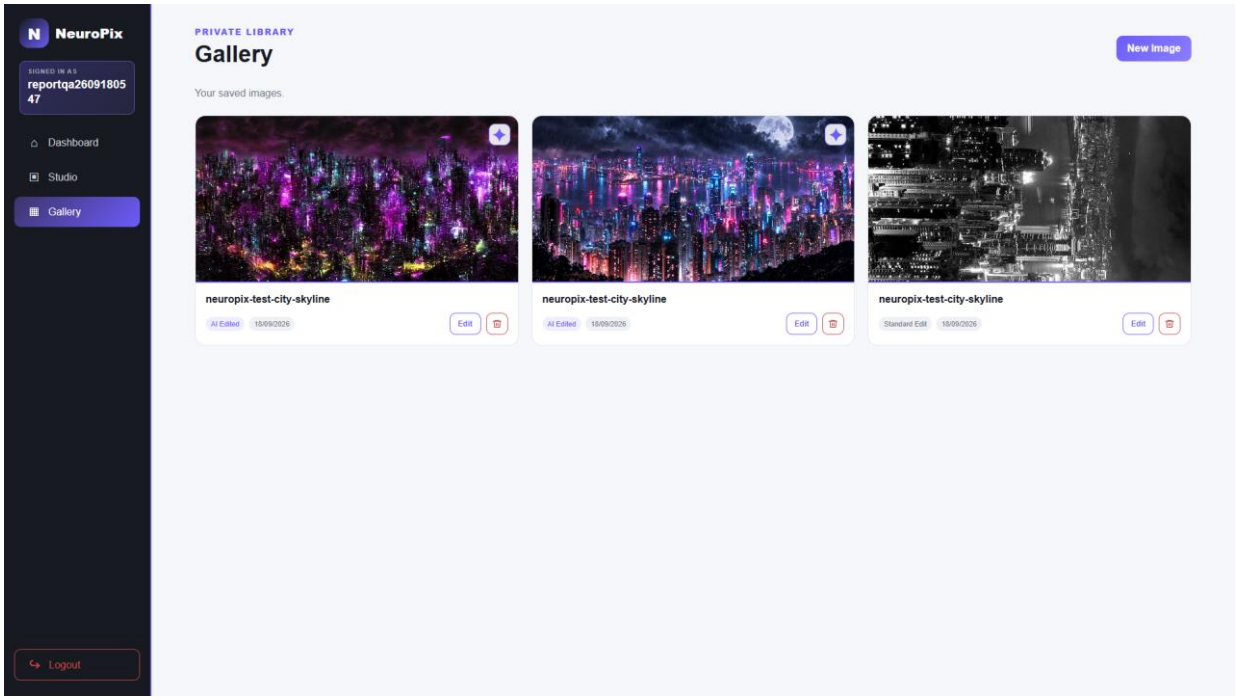


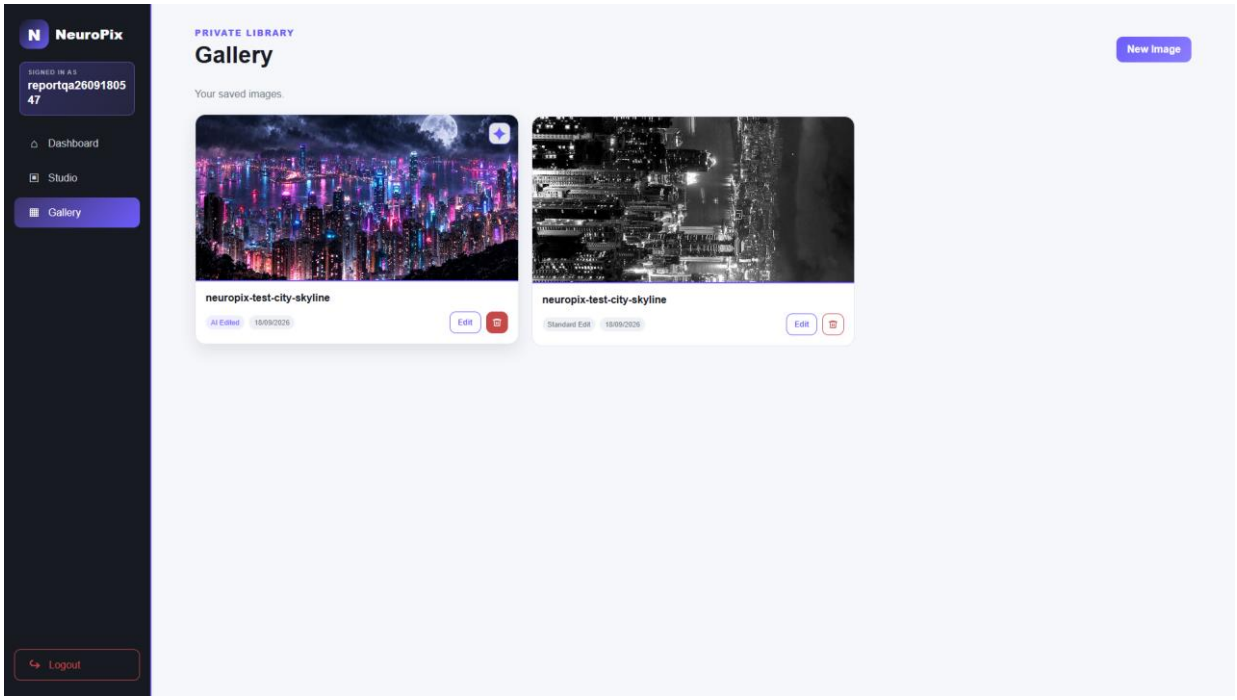
Figure 6.9-1: UC-T06 gallery verification and deletion sequence.

Table 6.9: UC-T06 execution result.

Step	Observed result
Open gallery	Three saved results appeared: two AI edits and one Standard edit
Delete latest result	Confirmation was accepted and the selected card was removed
Verify refresh	Two expected cards remained and no deleted card reappeared
Final status	**Pass**



UC-T06 (a) Saved image present in gallery



UC-T06 (b) Gallery after deletion

Figure 6.9-2: UC-T06 gallery evidence.

6.10 Results Summary and Evaluation

Table 6.10: Consolidated verification results.

Test	Scope	Result
Automated API and security checks	35 API, security, page, download, and mocked-persistence tests	Pass
UC-T01	Account registration	Pass
UC-T02	Login and protected dashboard	Pass
UC-T03	Upload, Standard processing, and comparison	Pass
UC-T04	Structured OpenAI edit and comparison	Pass
UC-T05	Local Stable Diffusion edit through the active tunnel	Pass
UC-T06	Gallery presence and deletion	Pass

All six live cases passed in the stated production environment. Standard editing produced the selected crop, rotation, and grayscale changes, OpenAI and local Stable Diffusion returned transformed images, and the deleted gallery item remained absent after refresh. Because the AI cases used one source and one set of instructions, the results support the integrated workflow rather than a comparison of model quality. The six documented live cases did not cover download or gallery re-editing, and testing did not measure performance under load, long-term reliability, or repeatability of AI output.

Chapter 7 Conclusion and Future Work

7.1 Achievement of the Project Aim

NeuroPix met its development aim by delivering a deployed web application for Standard and AI-assisted editing, comparison, download, and personal image management. The live tests supported the core editing and gallery workflows, while the automated checks covered selected backend behavior. We integrated these features within our budget and available hardware.

7.2 Critical Evaluation and Lessons Learned

A shared processing route allowed the same interface to support hosted and local AI. Their operating needs remained different, so the deployment had to account for both an external service and a computer maintained by the team.

The main trade-off was between operational simplicity and resilience. Synchronous requests were straightforward to integrate, but they leave users waiting for AI results. Hosting the local model on equipment we owned reduced cloud GPU costs, although availability depends on that computer and its tunnel. Coordinating S3 and MySQL also introduced partial-failure risks that the current implementation does not fully resolve.

Because some teammates were new to Git and GitHub, we worked directly on main to keep collaboration simple. Student offers and the RTX 3080 we already owned also helped us control costs, so these choices suited the size and budget of a student project. A larger team would benefit from feature branches, code review, a staging environment, and more managed deployment services to isolate changes and reduce operational dependencies.

The project required us to compare the finished application's behavior with our plans and reconsider providers when student offers did not cover our needs. We also learned that a completed AI request did not guarantee a useful result, so visual comparison remained necessary.

7.3 Recommended Six-Month Continuation

For another six months, we would tackle recovery and safer deployment first, then improve processing and evaluate the results. The priorities are:

- Improve recovery when image storage or database updates fail.
- Adopt feature branches, review, and staging.
- Move AI work to a recoverable background queue with progress reporting.
- Improve local output quality within RTX 3080 limits.
- Measure performance, accessibility, and cross-browser behavior.

7.4 Final Conclusion

NeuroPix brings Standard editing, guided AI requests, comparison, and saved results into one browser workspace. The project shows that this workflow can be delivered within our available hardware and budget.

References

[1] NeuroPix project team, “NeuroPix source repository,” GitHub, 2026. Accessed: Sep. 18, 2026. [Online]. Available: https://github.com/AbdulrahmanAlguydi/NeuroPix

[2] NeuroPix project team, “Software Requirements Specification,” unpublished project document, 2026.

[3] NeuroPix project team, “High-Level Design,” unpublished project document, 2026.

[4] Pillow contributors, “Pillow documentation.” Accessed: Sep. 18, 2026. [Online]. Available: https://pillow.readthedocs.io/en/stable/

[5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2022, pp. 10684-10695.

[6] OpenAI, “Introducing ChatGPT Images 2.5,” Sep. 8, 2026. Accessed: Sep. 18, 2026. [Online]. Available: https://openai.com/index/introducing-chatgpt-images-2-5/

[7] Adobe, “Generative AI features in Photoshop.” Accessed: Sep. 18, 2026. [Online]. Available: https://helpx.adobe.com/photoshop/desktop/generative-ai/generative-ai-features-overview.html

[8] Canva, “Edit photos with Canva photo editor.” Accessed: Sep. 18, 2026. [Online]. Available: https://www.canva.com/en_gb/help/image-editor/

[9] Amazon Web Services, “Amazon RDS for MySQL.” Accessed: Sep. 18, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_MySQL.html

[10] Pallets, “Security helpers,” Werkzeug documentation. Accessed: Oct. 9, 2026. [Online]. Available: https://werkzeug.palletsprojects.com/en/stable/utils/#security-helpers

[11] Pallets, “Sessions,” Flask documentation. Accessed: Oct. 9, 2026. [Online]. Available: https://flask.palletsprojects.com/en/stable/quickstart/#sessions

[12] stable-diffusion-v1-5, “Stable Diffusion v1-5 model card,” Hugging Face. Accessed: Oct. 9, 2026. [Online]. Available: https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5

[13] Hugging Face, “Image-to-image,” Diffusers documentation. Accessed: Oct. 9, 2026. [Online]. Available: https://huggingface.co/docs/diffusers/api/pipelines/stable_diffusion/img2img

[14] Amazon Web Services, “What is Amazon S3?” Amazon S3 User Guide. Accessed: Oct. 9, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

[15] F5, “NGINX reverse proxy,” NGINX documentation. Accessed: Oct. 9, 2026. [Online]. Available: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

[16] Electronic Frontier Foundation, “User guide,” Certbot documentation. Accessed: Oct. 9, 2026. [Online]. Available: https://eff-certbot.readthedocs.io/en/stable/using.html

[17] Amazon Web Services, “What is Amazon EC2?” Amazon EC2 User Guide. Accessed: Oct. 9, 2026. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html

[18] GitHub, “Self-hosted runners,” GitHub Docs. Accessed: Oct. 9, 2026. [Online]. Available: https://docs.github.com/en/actions/concepts/runners/self-hosted-runners

[19] Docker, “GPU support in Docker Desktop for Windows,” Docker Docs. Accessed: Oct. 9, 2026. [Online]. Available: https://docs.docker.com/desktop/features/gpu/

[20] OpenBSD, “ssh(1): OpenSSH remote login client,” OpenBSD manual pages. Accessed: Oct. 9, 2026. [Online]. Available: https://man.openbsd.org/ssh#R